



Using Synopsys Library IP in coreAssembler Tutorial

coreAssembler — Product Code: 3533-0

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History.....	5
Preface.....	9
About This Manual.....	9
Web Resources.....	9
Manual Overview.....	9
Tutorial Conventions.....	10
Synopsys Statement on Inclusivity and Diversity.....	10
Getting Help.....	10
Chapter 1 Introduction and Setup.....	13
1.1. Overview of Using Synopsys Library IP in coreAssembler.....	14
1.2. Installation and Setup.....	15
1.2.1. Licensing.....	15
1.3. Using coreAssembler Workspaces.....	16
1.4. Overview of coreAssembler Tutorials.....	17
1.5. Before You Begin.....	18
Chapter 2 Tutorial 1: Creating a Hierarchical Subsystem.....	19
2.1. Description.....	20
2.2. Objective.....	21
2.3. Before You Begin.....	22
2.4. Tutorial 1 Subsystem Design.....	23
2.5. Procedure 1: Create a New Workspace.....	24
2.6. Procedure 2: Specify Subsystem -- Add Components.....	26
2.7. Procedure 3: Specify Subsystem -- Memory Map Specification.....	37
2.8. Procedure 4: Specify Subsystem -- Configure Components.....	39
2.9. Procedure 5: Complete Connections.....	41
2.10. Procedure 6: Generate Subsystem RTL.....	44
Chapter 3 Tutorial 2: Creating a Testbench.....	47
3.1. Description.....	48
3.2. Objective.....	49

3.3. Before You Begin..... 50

3.4. Procedure 1: Generate a Workspace from a Batch Script.....51

3.5. Procedure 2: Creating a UVM (RAL Subsystem) Testbench Workspace..... 52

 3.5.1. Tutorial 2 Summary.....53

Chapter 4 Tutorial 3: Synthesizing Your Design.....55

 4.1. Description..... 56

 4.2. Objective..... 57

 4.3. Procedure 1: Recreating a Workspace from Batch..... 58

 4.4. Tutorial 3 Summary.....65

Chapter 5 Menus and Toolbars..... 67

Revision History

Version	Date	Description
W-2024.09-1	October 2024	Updated version and date
W-2024.09	September 2024	Updated version and date
V-2024.06-2	June 2024	Updated version
V-2024.06	June 2024	■ Version and date update
V-2024.03-1	May 2024	■ Version and date update
V-2024.03	March 2024	■ Changed instances of DesignWare Cores to Synopsys IP to align with corporate naming conventions.
V-2023.12-1	January 2024	■ Version and date update
V-2023.12	December 2023	■ Version and date update
V-2023.09-1	September 2023	■ Version update
V-2023.09	September 2023	■ Version and date update
U-2023.06-1	June 2023	■ Version update
U-2023.06	June 2023	■ Version and date update
U-2022.12-2	February 2023	■ Version and date update
U-2022.12	December 2022	■ Version and date update
T-2022.06-SP2	September 2022	■ Version and date update
T-2022.06-SP1	July 2022	■ Version and date update
T-2022.06	June 2022	■ Version and date update

Version	Date	Description
S-2021.09-SP4	April 2022	■ Version and date update
S-2021.09-SP3	March 2022	■ Updated “Procedure 2: Creating a UVM (RAL Subsystem) Testbench Workspace” on page 37 ■ Updated “Procedure 1: Create a New Workspace” on page 15
S-2021.09-SP2	January 2022	■ Version and date update
S-2021.09-SP1	October 2021	■ Version and date update
S-2021.09	September 2021	■ Version and date update
R-2020.12-SP2	March 2021	■ Added Select Items menu choice to Table 5-1 on page 47
Q-2020.03-SP1	April 2020	■ To ensure success when running the simulation script, added step 10 in “Procedure 2: Creating a UVM (RAL Subsystem) Testbench Workspace” on page 37
P-2019.06-SP1	July 2019	■ Updated the format of this document ■ Updated all interface screenshots
P-2019.06	June 2019	In “Tutorial 2: Creating a Testbench” on page 35, provided procedures for a UVM testbench instead of a VMM testbench
J-2014.12	December 2014	Version change. Updated tutorial flow and some screenshots.
J-2014.09	September 2014	Version change. Removed ‘Define Interface’ from coreAssembler GUI.
I-2013.09	September 2013	Platform and version change. Removed VMT testbench creation.
H-2013.03	March 2013	Updates to screenshots and some tutorial steps.
G-2012.06-SP3	January 2013	Update names of DW_apb_uart and DW_apb_timers, page 22. New Copyright page. Improve VMT flow, page 22.
G-2012.06-SP2	October 2012	Date / Version update. Updated screenshots. New doc template. Update links. Added clarifying text. Removed obsolete usage table.
G-2012.06	June 2012	Date / Version update. Updated graphics and Appendix A menu items.
F-2011.12	December 2011	Date / Version update. Replaced “Perform ASIC Synthesis” window graphic on page 44 (new default Design_Compiler_Reference_Methodology)
F-2011.06	June 2011	STAR fixes; Major GUI updates (merging to Specify Subsystem activity and tabbed sub-activities); Many updated figures.
E-2010.09	September 2010	STAR fixes; GUI updates (Technology dialog and report, Enable Activity); Activity changes (Perform ASIC Synthesis)
D-2010.03	March 2010	GUI updates (Complete Connections, Change Connections, etc.)

Version	Date	Description
C-2009.06-SP2	January 2010	Copyright update.
C-2009.06-SP1	October 2009	Activity reports and design views are not automatically generated. New Reports tab behavior. Complete Connections window reorganized. Export interface dialog has new naming “format” options.
C-2009.06	June 2009	Complete tutorial rewrite. Simplified overview material. Created 3-part tutorial: Create RTL, Verify subsystem, Synthesize.
B-2008.12-SP1	January 2009	Changed Big- to Little-Endian in tutorial to fix erroneous simulation results.
B-2008.12	December 2008	Updated Screenshots, Menu Items, Activity List, Synthesis Strategies.
B-2008.06	June 2008	Updated Screenshots, Memory Map, Slave Visibility, and so on.

Preface

This chapter contains the following sections:

- [“About This Manual”](#) on page 9
- [“Web Resources”](#) on page 9
- [“Manual Overview”](#) on page 9
- [“Tutorial Conventions”](#) on page 10
- [“Synopsys Statement on Inclusivity and Diversity”](#) on page 10
- [“Getting Help”](#) on page 10

About This Manual

This manual contains a tutorial and information about using Synopsys Library IP. This manual is intended for designers and integrators who want to learn how to quickly assemble an ARM® AMBA® subsystem using Synopsys Library IP in coreAssembler. AMBA is a registered trademark of Arm Limited and is used under license.

This manual does not contain exhaustive information about coreAssembler. For more detailed information about coreAssembler, refer to the *coreAssembler User Guide*.

Web Resources

- <http://www.synopsys.com/Support/LI> - Synopsys® Licensing information
- <http://www.designware.com> - Synopsys IP downloads and documentation
- <http://www.synopsys.com> - General information about Synopsys and its products
- <https://solvetplus.synopsys.com/> - SolvNetPlus site for customer support

Manual Overview

This manual contains the following chapters:

Table 1 Manual Overview

“Introduction and Setup” on page 13	Give details of Synopsys Library IP in coreAssembler to get you using the tool.
---	---

Table 1 Manual Overview (continued)

“Tutorial 1: Creating a Hierarchical Subsystem” on page 19	Shows how to create a multi-layer subsystem that consists of the following Synopsys synthesizable IP components: DW_axi, DW_axi_x2h, DW_ahb, DW_apb, DW_ahb_dmac, DW_ahb_ictl, DW_apb_timers, and DW_apb_uart.
“Tutorial 2: Creating a Testbench” on page 47	Shows how to create a UVM testbench for your subsystem design and then run the simulation for it.
“Tutorial 3: Synthesizing Your Design” on page 55	Shows how to create a new subsystem workspace using a batch script, prepare for the Synthesize activity, and perform a Synthesis run.
“Menus and Toolbars” on page 67	Provides a table of toolbar icons and corresponding menu choices to accomplish subsystem assembly.

Tutorial Conventions

Each tutorial in this document is structured with the following topics:

- **Description** – briefly describes the design you are going to create
- **Objective** – highlights the specific tasks you will perform to create the tutorial design
- **Tutorial Procedures** – steps you through all tasks to complete the tutorial
- **Tutorial Summary** – summarizes the activities you completed during the tutorial

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Getting Help

If you have a question about using Synopsys products, please consult product documentation that is installed on your network. You can also access documentation for Synopsys products on the Web:

- Datasheets for individual Synopsys IP components, located using “Search for IP”:
<https://www.synopsys.com/designware-ip.html>
- License Information -- Synopsys Common Licensing Quickstart:
<http://www.synopsys.com/Support/LI>

You can obtain product support in the following ways:

- Go to SolvNet to submit support requests and access documentation and software downloads:
<https://solvnetplus.synopsys.com/>
- Send an e-mail message to http://support_center@synopsys.com.
- Global Support Centers -- contact information and hours of operation:
<https://www.synopsys.com/support/global-support-centers.html>

1

Introduction and Setup

This chapter provides an overview of the Synopsys Library IP in coreAssembler, which can help you accelerate your SoC design flow. This chapter also describes setup and other general information to get you started using Synopsys Library IP in coreAssembler.

This chapter contains the following sections:

- [“Overview of Using Synopsys Library IP in coreAssembler”](#) on page 14
- [“Installation and Setup”](#) on page 15
- [“Using coreAssembler Workspaces”](#) on page 16
- [“Overview of coreAssembler Tutorials”](#) on page 17
- [“Before You Begin”](#) on page 18

1.1 Overview of Using Synopsys Library IP in coreAssembler

The coreAssembler tool is a highly flexible, integrated and feature-rich design environment that allows you to select, configure, interconnect, simulate, and synthesize Synopsys IP for AMBA[®] 2 and AMBA 3 AXI. AMBA is a registered trademark of Arm[®] and is used under license.

Synopsys provides coreAssembler and coreConsultant, both from the Synopsys coreTools suite, for the purposes of configuration, synthesis, and verification of synthesizable IP. For information on the different coreTools applications, refer to *Guide to coreTools Documentation*.

A *workspace* is your custom-configured version of a component or subsystem, where you connect, configure, simulate, and synthesize your implementation. coreConsultant is the basic tool used to create a workspace for a single IP component. The coreAssembler tool allows you to create a workspace with *multiple* IP components connected within a subsystem.

1.2 Installation and Setup

You must install and properly set up all the components and tools used with the coreAssembler tool before you can create a subsystem using Synopsys Library IP.

- For coreTools (including coreAssembler) installation instructions, see “Downloading and Installing the Release Image” in the *coreTools Release Notes*.
- For Synopsys Library IP (AMBA) installation instructions and information about required tools and versions, see “Supported Versions of Tools and Libraries” in the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

1.2.1 Licensing

Table 1-1 describes the required licenses for coreAssembler.

Table 1-1 Required Licenses for IP in coreAssembler

Type of IP	Required License	Usage Documentation
DW Library IP	DesignWare (or coreAssembler)	<i>coreAssembler User Guide</i>
Other coreBuilder packaged IP	coreAssembler (or coreBuilder*)	<i>coreAssembler User Guide</i>
Non- DW IP or IP-XACT IP	coreAssembler	<i>coreAssembler User Guide</i>
* Note: coreAssembler can use a coreBuilder license for DesignWare and coreKit IP. Synthesis does not require a DesignWare license if source code license and source for DW Building Blocks is used.		

For a more details or for licensing of other tools within coreTools family, refer to “Licensing” in the *coreTools Release Notes*.

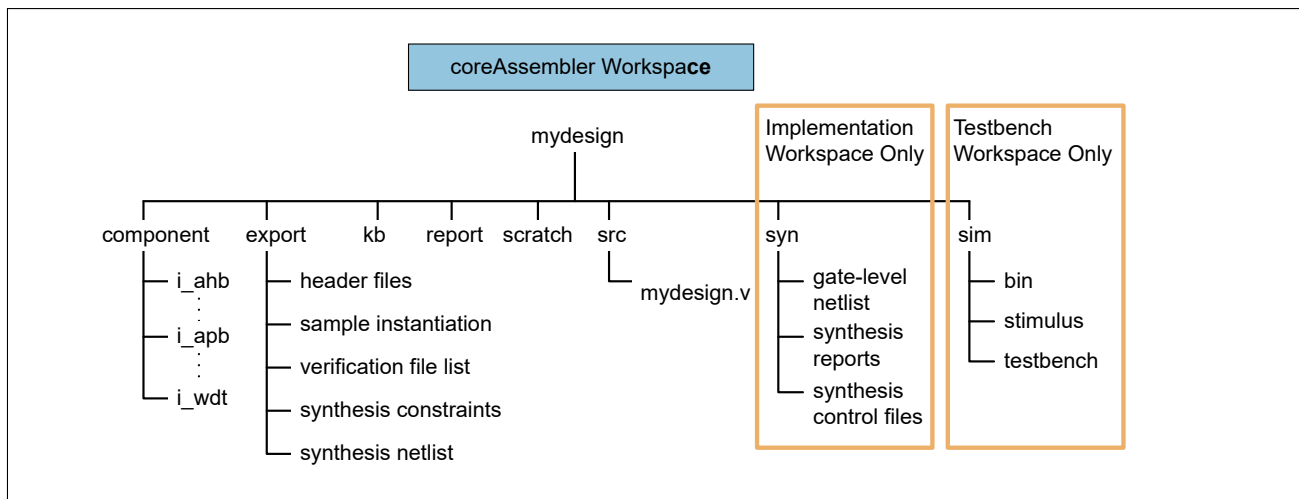
1.3 Using coreAssembler Workspaces

A workspace is your custom-configured version of a subsystem where you connect, configure, simulate, and synthesize your subsystem. Workspaces contain directories, files, and symbolic links to IP component installations (in \$DESIGNWARE_HOME for DesignWare Library IP). Workspaces are of two types:

- Subsystem workspace – this is the workspace for your synthesizable IP design
- Testbench workspace – this is where you build a testbench around your Device Under Test (DUT) and simulate your subsystem.

Figure 1-1 shows the directories for both types of coreAssembler workspaces. Some of the directories are created after you perform the related activities (for example, the export and syn (synthesis) directories).

Figure 1-1 coreAssembler Workspace Tree



1.4 Overview of coreAssembler Tutorials

The tutorials in this chapter are designed to get you quickly up-to-speed on how to use coreAssembler to create your Synopsys SoC subsystem. Each tutorial shows specific tasks that you will encounter when you begin using the tool to build your own design. The tutorials are as follows:

- [“Tutorial 1: Creating a Hierarchical Subsystem”](#) – Provides the necessary steps to create a hierarchical subsystem design, configure the design, create a memory map, and generate the subsystem RTL.
- [“Tutorial 2: Creating a Testbench”](#) – Uses the Tutorial 1 design as the device under test (DUT), and steps you through the process of creating a testbench workspace, adding verification models, performing a simulation run, and saving the testbench code.
- [“Tutorial 3: Synthesizing Your Design”](#) – Uses the Tutorial 1 design, and guides you through the steps to set up and perform a synthesis run.

1.5 Before You Begin

It is critical that you have correctly installed and set up coreAssembler and your Synopsys Library IP environment to use Synopsys Library IP in the coreAssembler tool. Please refer to the installation information in the *coreTools Release Notes* and the *DesignWare Synthesizable Components for AMBA 2/AMBA 3 AXI Installation Guide*.

2

Tutorial 1: Creating a Hierarchical Subsystem

This chapter contains the following sections:

- [“Description”](#) on page 20
- [“Objective”](#) on page 21
- [“Before You Begin”](#) on page 22
- [“Tutorial 1 Subsystem Design”](#) on page 23
- [“Procedure 1: Create a New Workspace”](#) on page 24
- [“Procedure 2: Specify Subsystem -- Add Components”](#) on page 26
- [“Procedure 3: Specify Subsystem -- Memory Map Specification”](#) on page 37
- [“Procedure 4: Specify Subsystem -- Configure Components”](#) on page 39
- [“Procedure 5: Complete Connections”](#) on page 41
- [“Procedure 6: Generate Subsystem RTL”](#) on page 44

2.1 Description

In this tutorial, you will create a multi-layer subsystem that consists of the following Synopsys synthesizable components for AMBA 2/AMBA 3 AXI:

- DW_axi
- DW_axi_x2h
- DW_ahb
- DW_apb
- DW_ahb_dmac
- DW_ahb_ictl
- DW_apb_timers
- DW_apb_uart

2.2 Objective

The purpose of this tutorial is to show the coreAssembler activities used for subsystem RTL creation using the following “Create RTL” group of activities:

- [“Procedure 1: Create a New Workspace”](#) on page 24
- [“Procedure 2: Specify Subsystem -- Add Components”](#) on page 26
- [“Procedure 3: Specify Subsystem -- Memory Map Specification”](#) on page 37
- [“Procedure 4: Specify Subsystem -- Configure Components”](#) on page 39
- [“Procedure 5: Complete Connections”](#) on page 41
- [“Procedure 6: Generate Subsystem RTL”](#) on page 44

You will need to complete all of the steps in this tutorial in order to begin Tutorial 2.

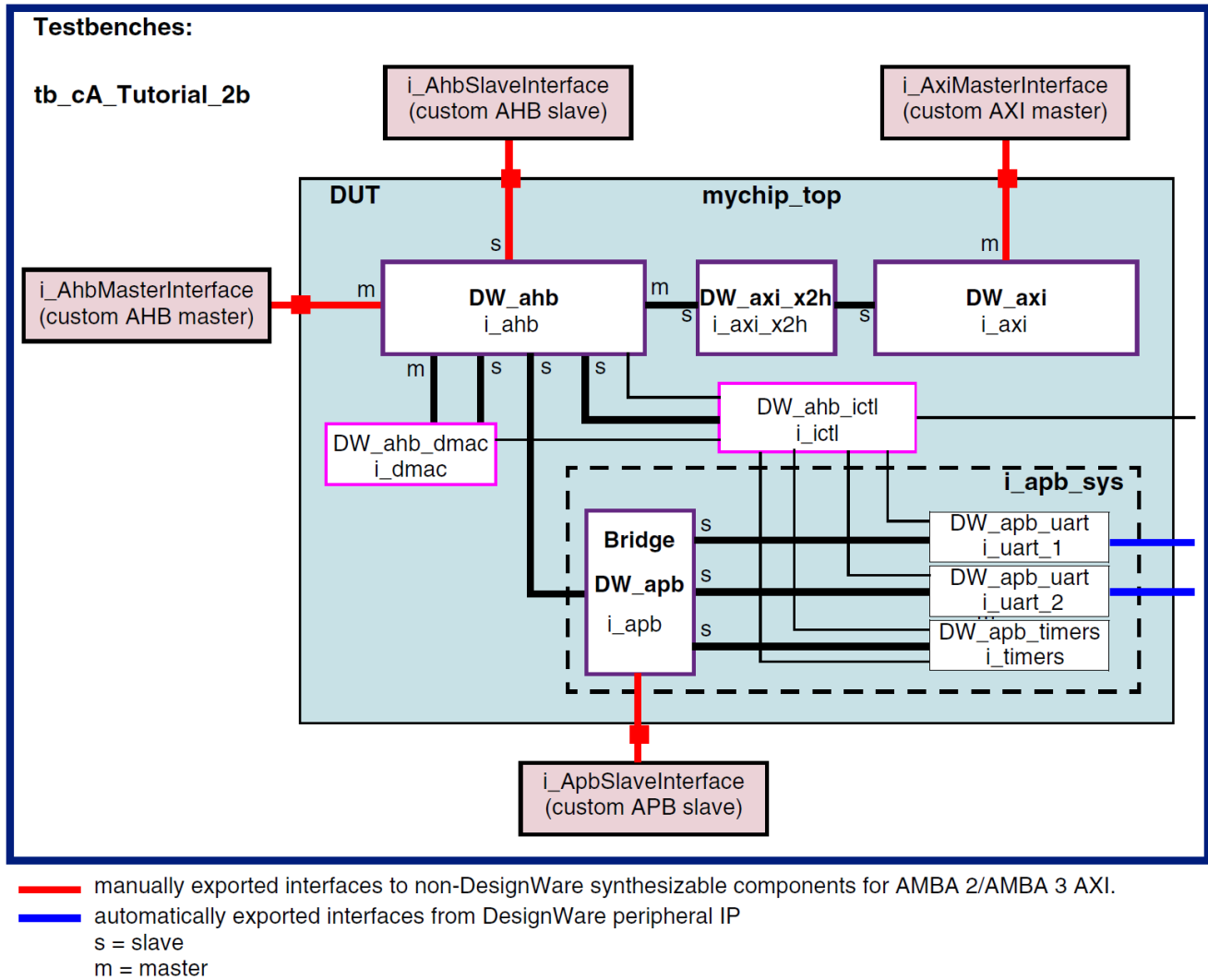
2.3 Before You Begin

Because the tutorials are streamlined to help you complete them in minimum time, some general information about the coreAssembler tool is not provided here. In case you want more information, links to the appropriate section in the *coreAssembler User Guide* are provided throughout the tutorial.

2.4 Tutorial 1 Subsystem Design

Figure 2-1 shows the layout of the design that you create in Tutorial 1. It contains a hierarchical APB block, and both AXI and AHB buses. A DMA controller provides direct access to external memory.

Figure 2-1 Tutorial 1 Subsystem Block Diagram



2.5 Procedure 1: Create a New Workspace

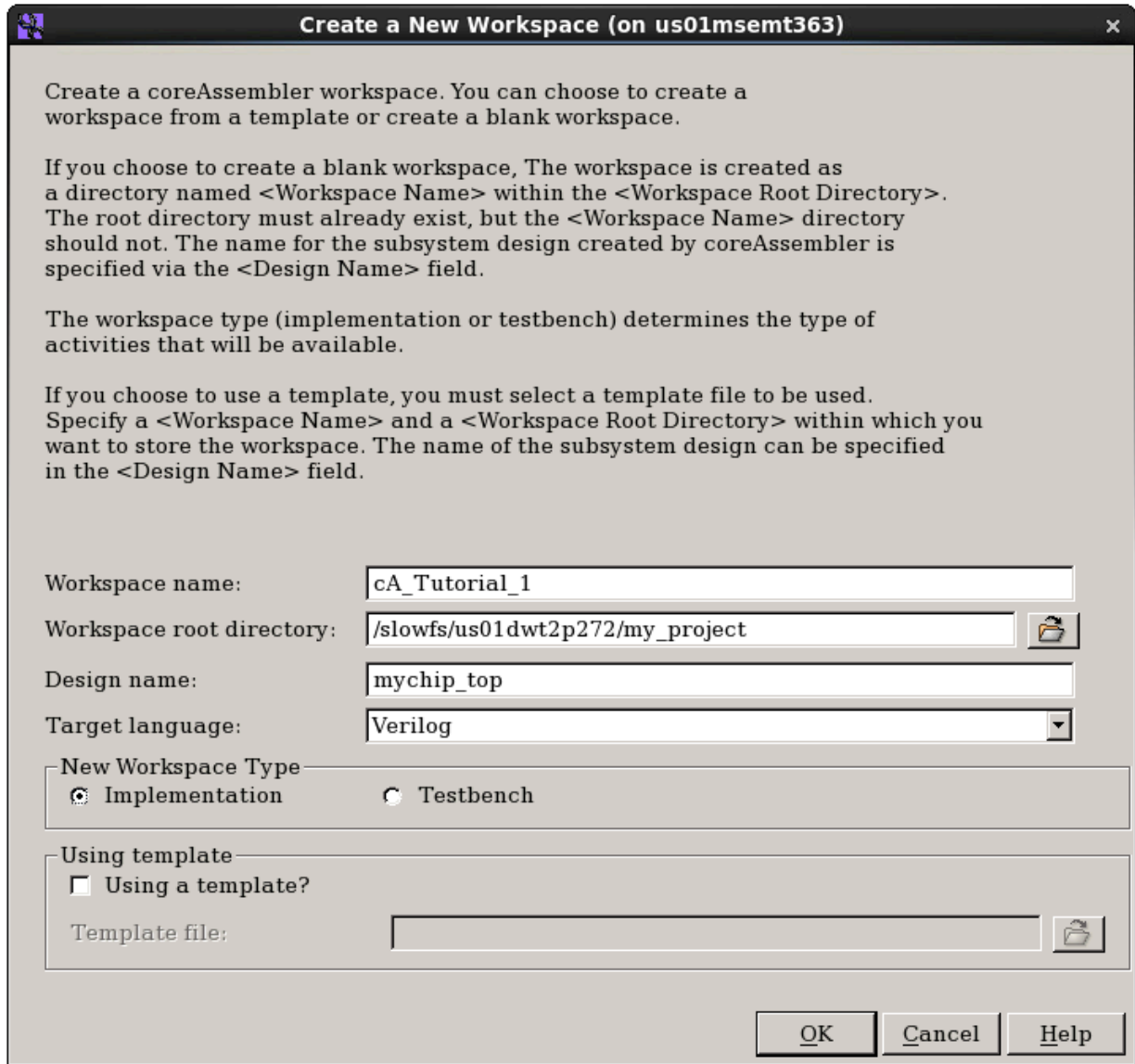
Complete the following steps:

1. Invoke the coreAssembler environment.

% **coreAssembler**

2. In the main menu, click on the **File > New Workspace** menu item to display the “Create a New Workspace” dialog:

Figure 2-2 Creating a New Workspace



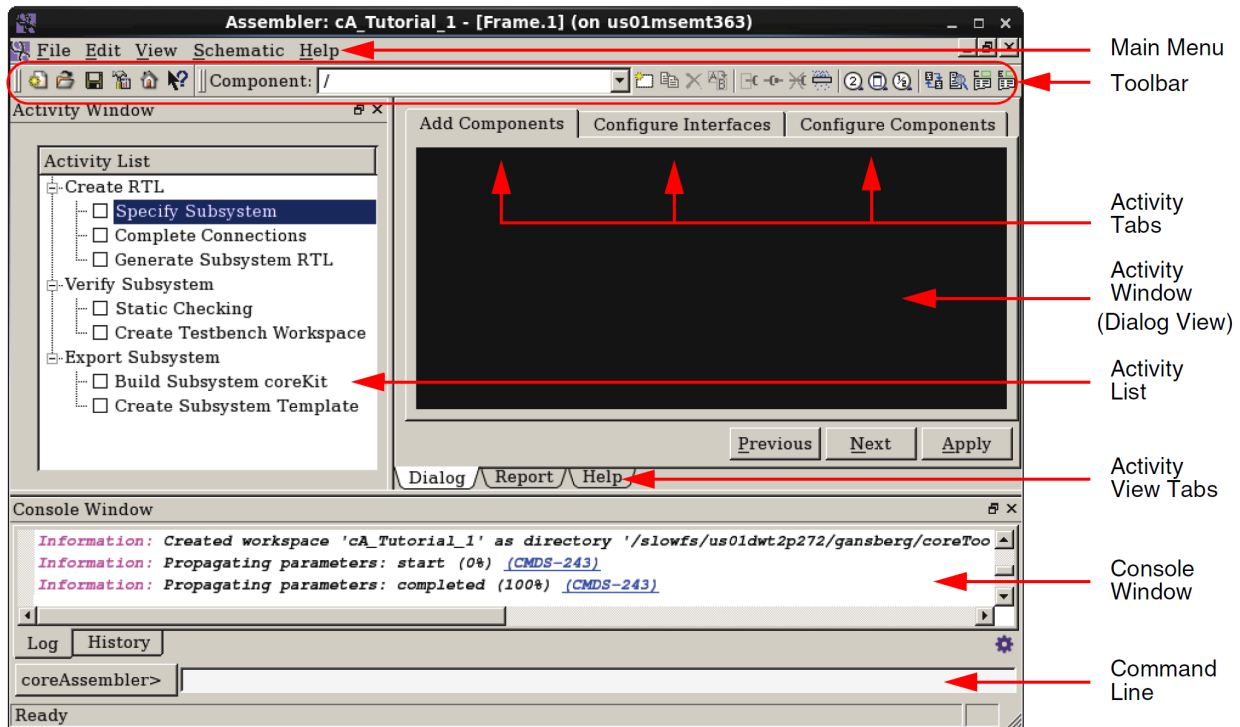
3. Set the following fields and then click **OK**:

Table 2-1 Tutorial 1 Workspace Naming

Field Name	Value
Workspace name (unique name, of your choice, of Unix directory to contain all files)	cA_Tutorial_1
Design name (top-level design name of your choice; does not need to be unique)	mychip_top
Target language	Verilog

4. The Add Components activity tab should be active (if not, click on the Add Components activity tab).

You should see a window layout similar to the following figure. The tabs at the top of the Activity View pane allow you to easily move between adding components, making configuration changes, and updating a memory map.

Figure 2-3 Assembler Activity View Pane

To see the function of each toolbar icon, hover your mouse over the icon.

**Note**

Some activity tabs, such as the Memory Map activity tab, are added when a placed component might require the settings in the tab. This is observed in Step 2 in the next procedure.

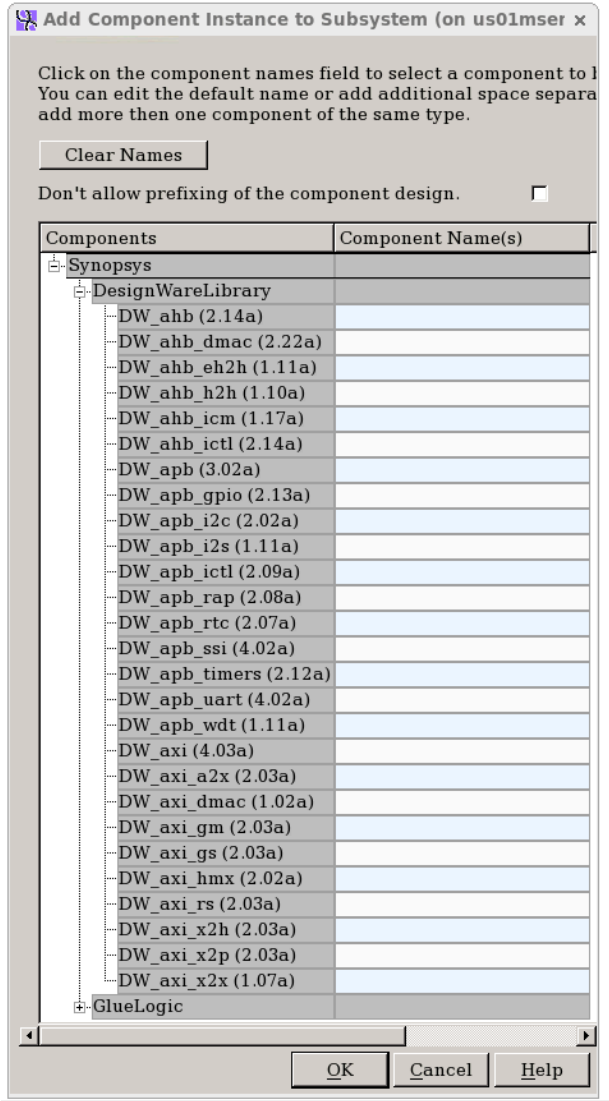
2.6 Procedure 2: Specify Subsystem -- Add Components

1. From the **Add Components tab view**, right-click in the Activity Window to show the schematic menu, then choose **Add New Component**.
- Or click the icon on the toolbar as seen in [Figure 2-4](#)

Figure 2-4 Add New Component Button



Figure 2-5 Add New Component Window

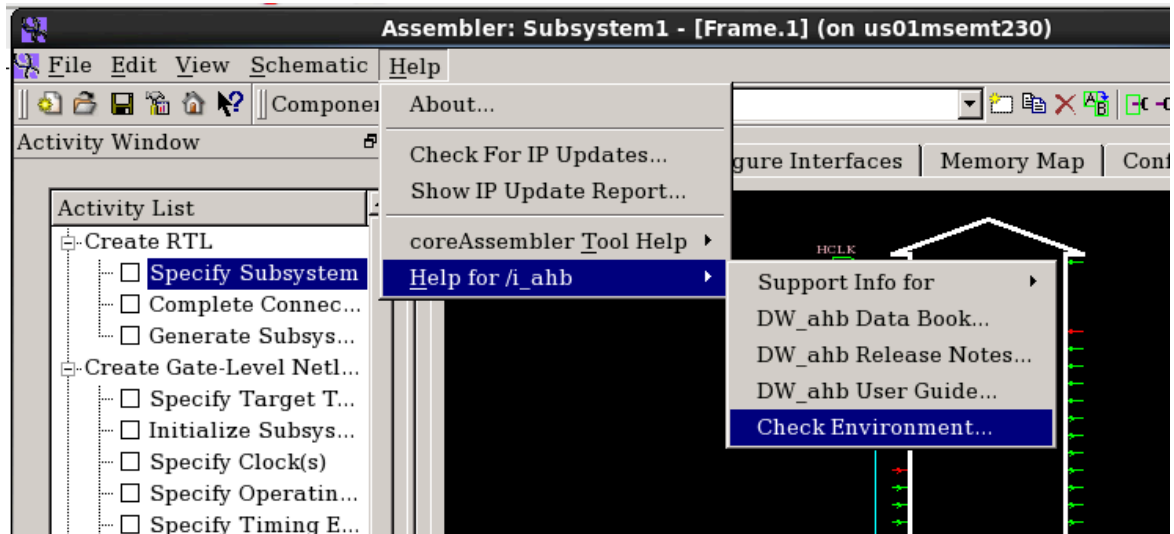


2. From the list of available components, click on the **DW_ahb** component.
- The default instance name (**i_ahb**) appears in the Component Name(s) column. You can click on the instance name to edit the default, but for this step, use the default name. Click OK.
- When the component is added, the Schematic View of a single DW_ahb component is shown. Also note that the Memory Map activity tab now appears.

- Before you continue, confirm that you have properly set up your environment and have installed the supported versions of tools (such as simulators, Design Compiler, and so on).

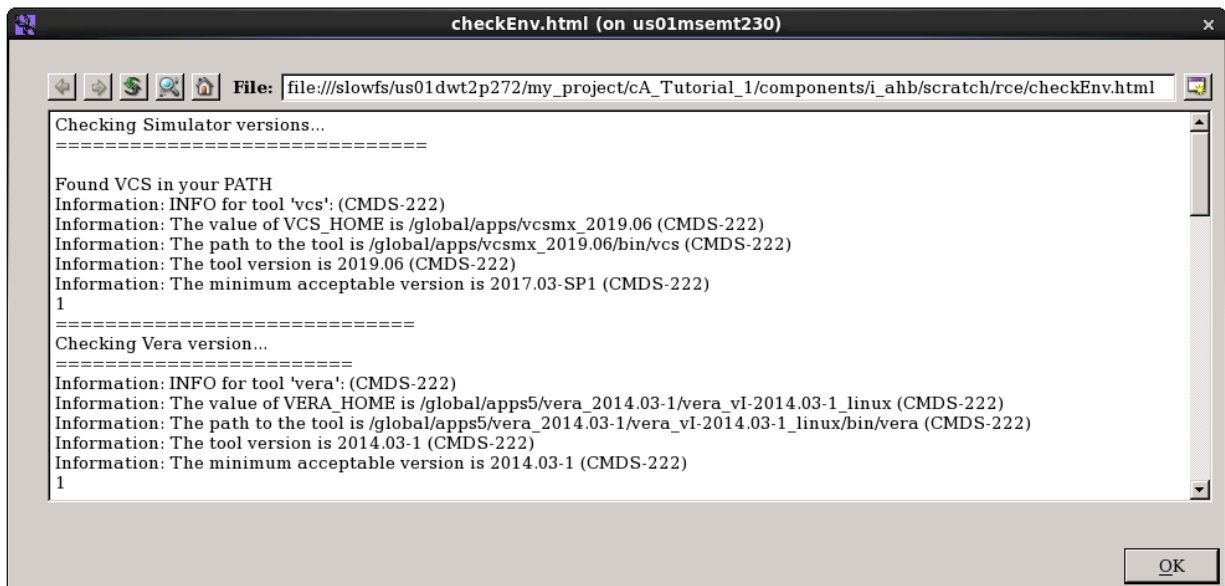
In the **Help** menu, select **Help for /i_ahb > Check Environment**. (Not available in J2014.12)

Figure 2-6 Specify Subsystem – Check Environment



A report similar to the following figure is displayed. Each section in the report lists a tool, whether it is installed, and whether it passes the checks. A “1” in the last line of a report section indicates the tool is present and passes the checks. A “0” appears if a tool does not pass the checks.

Figure 2-7 Check Environment Window



**Note**

If you see warnings or errors in this report, you must correct them before continuing on to Tutorial 2 and Tutorial 3. For information about the requirements, see [“Installation and Setup”](#) on page 15 “Installation and Setup” on page 9.

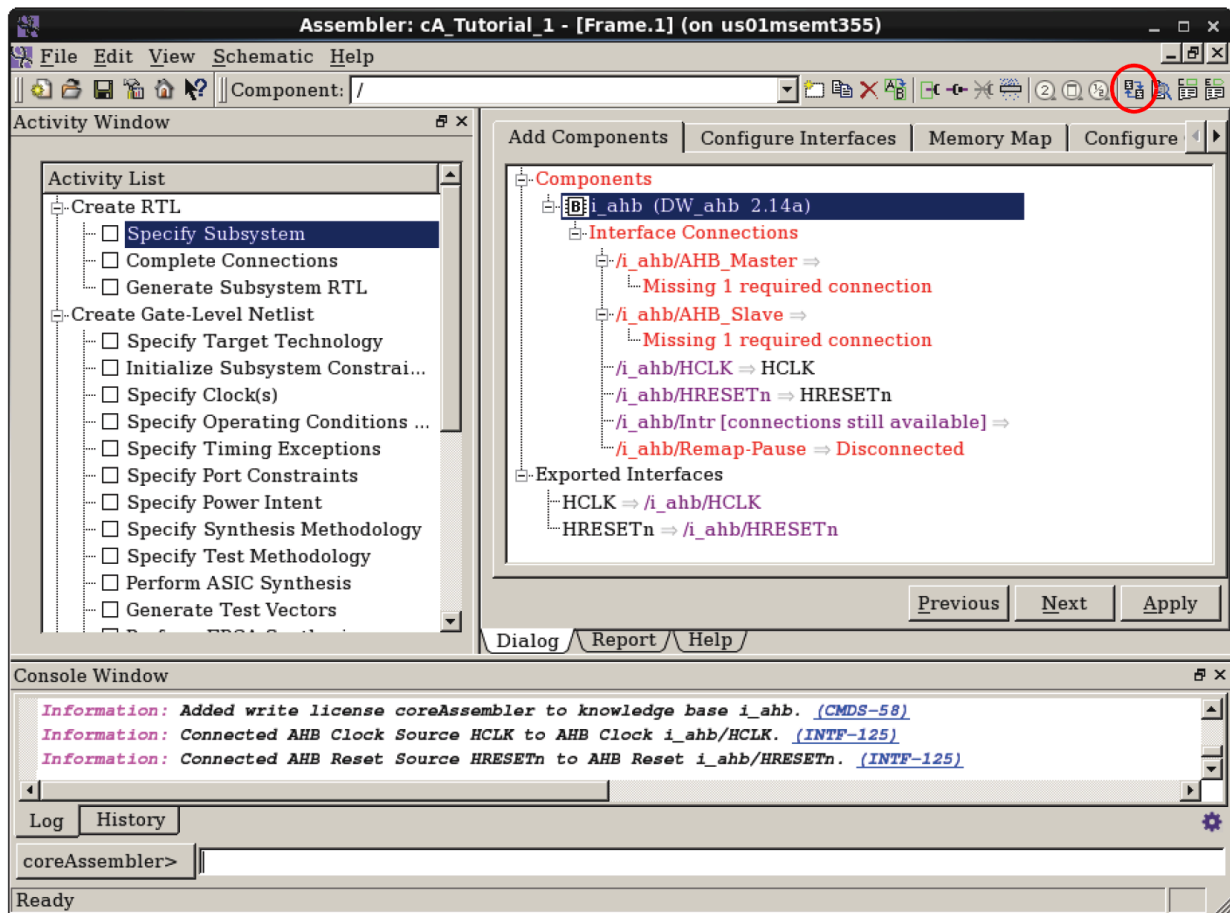
After resolving any errors/warnings, run the Design IP Environment Check again to confirm your environment and tools meet requirements.

- In addition to the schematic view, the coreAssembler tool provides a hierarchical tree listing of the design. Inspect the tree view by clicking the Toggle View toolbar icon, circled in red as seen in [Figure 2-8](#).

Figure 2-8 Toggle View Button



Figure 2-9 Specify Subsystem – Toggle View



Here, you see a text view of the components and connections in the schematic. Elements with hierarchy can be expanded and collapsed. Red text alerts you to missing interface connections. As you build the subsystem in this tutorial, you will resolve these issues.

- Toggle back to the schematic view. Note that components outlined in red need interface connections.

6. Add the following components as you did in Step 1. You can use CTRL-click to select and add multiple components in one step.

- ☐ DW_axi_x2h
- ☐ DW_axi

When the Interface Connection Dialog box appears, which allows you to make further connections, simply OK the dialog because you will complete the connections later.

When these components are added, connections are automatically made from the AXI Slave through the X2H component to the AHB_Master1 interface. Clock and reset connections are also made automatically.

7. Add the DW_ahb_dmac component.

The DW_ahb_dmac component connects to a Master port and a Slave port on the DW_ahb component. The combined interrupt line on the DW_ahb_dmac component is not automatically connected. Next, you add an interrupt controller to make the necessary interrupt connections.

8. Add the DW_ahb_ictl component.

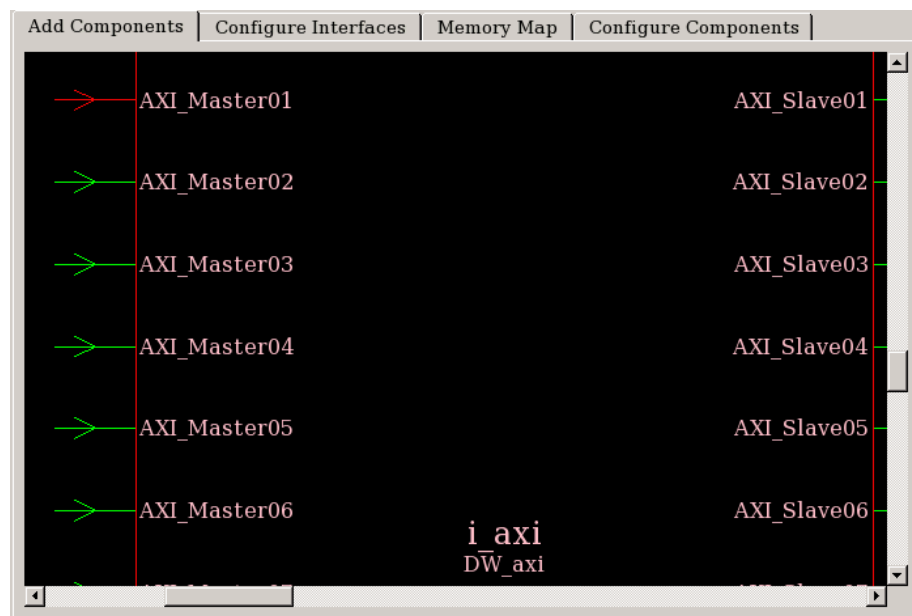
This component is added and shows a large number of interrupt ports available. You won't need all of these ports, so next you need to configure ICTL for the correct number of ports.

9. Zoom in on the i_axi instance so you can locate the AXI_master01 port.

In the popup menu, select **Zoom > Zoom In 2x**, or you can press CTRL =, or you can click . Magnification can also be controlled by the mouse scroll wheel, if available.

The pin for AXI_Master01 is red, indicating that it is unconnected.

Figure 2-10 Add Component Window



10. Right-click on the AXI_Master01 input port and choose Export Interface. Click OK, which keeps the default export name.

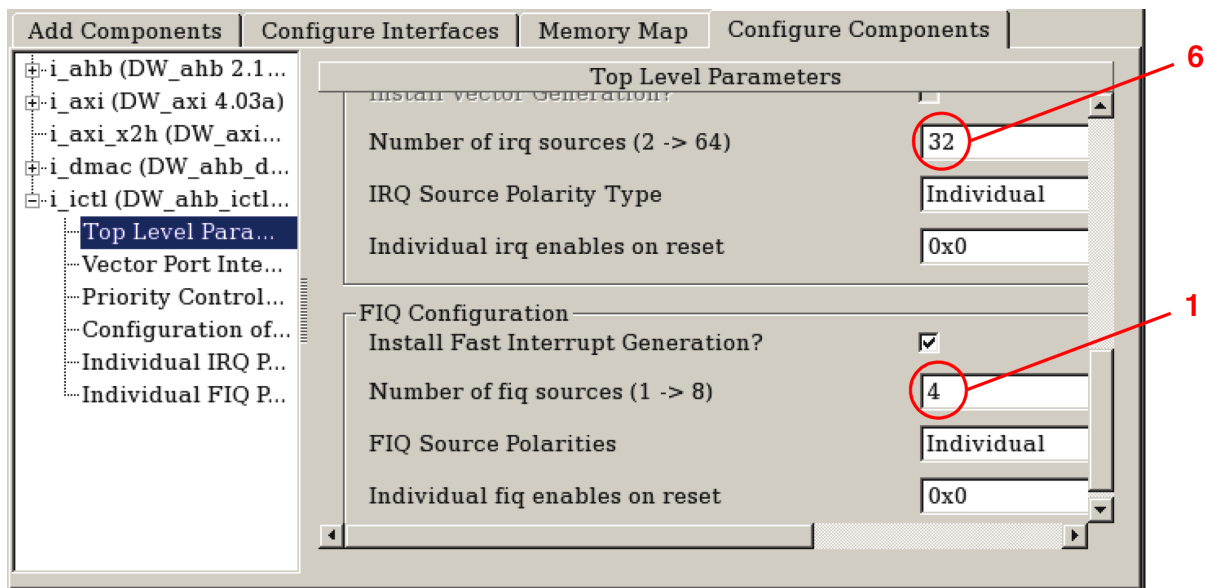
This action automatically creates a port on the schematic and makes the appropriate connections to the interface being exported. The `i_axi` instance turns from red to yellow, indicating that no interface issues remain. Also, the magnification is automatically set to Zoom Full to show the full schematic as seen in [Figure 2-11](#).

Figure 2-11 Magnification Button



11. In a similar manner select and export the CPU output port on the `i_ictl` instance. Use default naming. The `i_ictl` component is now green, indicating that there are no issues.
12. Finally, locate the Remap-Pause input port near the top of the `i_ahb` instance. But, instead of exporting the interface, right-click and choose **Set Unused**. The outline of the `i_ahb` instance turns yellow.
13. Click on the Configure Components activity tab to show a list of the components in the schematic.
14. Expand the listing of the `i_ictl` instance and click on the Top-level Parameters item. Scroll down in the Top Level Parameters list and change the number of “irq” and “fiq” sources as shown in the diagram below. Do not click Apply yet because you have not completed all connections.

Figure 2-12 Configure Component Window



15. Click on the **Add Components** activity tab. You will see the Apply Configure Components Changes dialog. Click **OK** to apply the changes that you made in the Configure Components view.

The `i_ictl` instance now shows significantly fewer interrupt ports. You will connect the `i_ictl` interrupts later, after all of the components have been added.

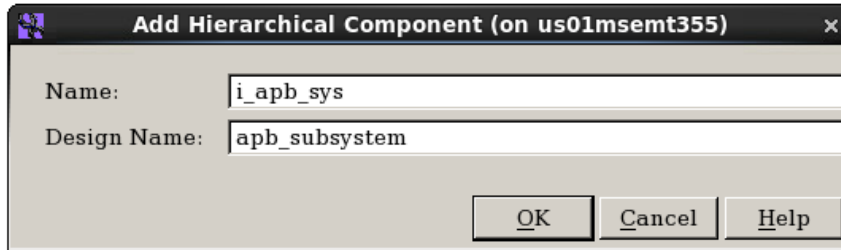
To refresh your memory of the system you are creating, see [Figure 2-1](#) for reference.

Next, you will build the `DW_apb` and peripherals in a hierarchical cell. But, before you do this, you should save your work, as follows:

16. From the File menu, choose “Save Workspace.” Your work to this point is now saved in the `cA_Tutorial_1` workspace directory.

17. To begin adding the DW_apb and peripherals in a hierarchical cell, right-click in the Schematic window, and choose **Add Hierarchical Level**. In the dialog box, enter the names that appear below and click OK.

Figure 2-13 Add Hierarchical Component Window



The coreAssembler tool creates the cell and automatically puts you at the new level of hierarchy with a blank schematic (the previous schematic view is replaced). To move up and out of a hierarchical level, use the **Exit cell** popup menu item. To descend into a cell hierarchy, simply double-click on the block.

18. Add the following components in this level of hierarchy. To rename an instance, click on the default name in the **Component Name(s)** column.

- ☐ dw_apb (i_apb)
- ☐ dw_apb_uart (i_uart_1) (rename this instance "i_uart_1" before adding)
- ☐ dw_apb_uart (i_uart_2) (rename this instance "i_uart_2" before adding)
- ☐ dw_apb_timers (i_timers)

This can be done in two steps:

- a. Add DW_apb, DW_apb_uart (the i_uart_1 instance), and DW_apb_timers in one step,
- b. Then add the i_uart_2 instance of DW_apb_uart.

19. Examine the results in the schematic window. Notice that the i_uart_1 output is named "SIO" and the i_uart_2 output is named "i_uart_2_SIO." To keep naming consistent, do the following:

- a. On the i_uart_1 instance, select and remove the exported "SIO" port symbol (using Remove Item in the popup menu)
- b. Select this port again, choose **Export Interface** in the popup menu.
- c. Specify the exported name as "i_uart_1_SIO".

Using <instance_name>_<signal_name> establishes a consistent naming convention.

20. Export these interfaces out of the i_apb_sys subsystem using the Schematic menu:

- ☐ i_apb_sys/i_apb/AHB_Slave (name = AHB_Slave)
- ☐ i_apb_sys/i_apb/HCLK (name = HCLK)
- ☐ i_apb_sys/i_apb/HRESETn (name = HRESETn)
- ☐ i_apb_sys/i_uart_1/Intr (default name = ex_i_uart_1_Intr)
- ☐ i_apb_sys/i_uart_2/Intr (default name = ex_i_uart_2_Intr)

- i_apb_sys/i_timers/Intr0 (default name = ex_i_timers_Intr0)

- i_apb_sys/i_timers/Intr1 (default name = ex_i_timers_Intr1)

Make sure that none of the components are still outlined in red, indicating missing connections.

21. Before exiting the cell, first save your work.

22. Right-click in the schematic view and click on **Exit Cell** to move to the top level.

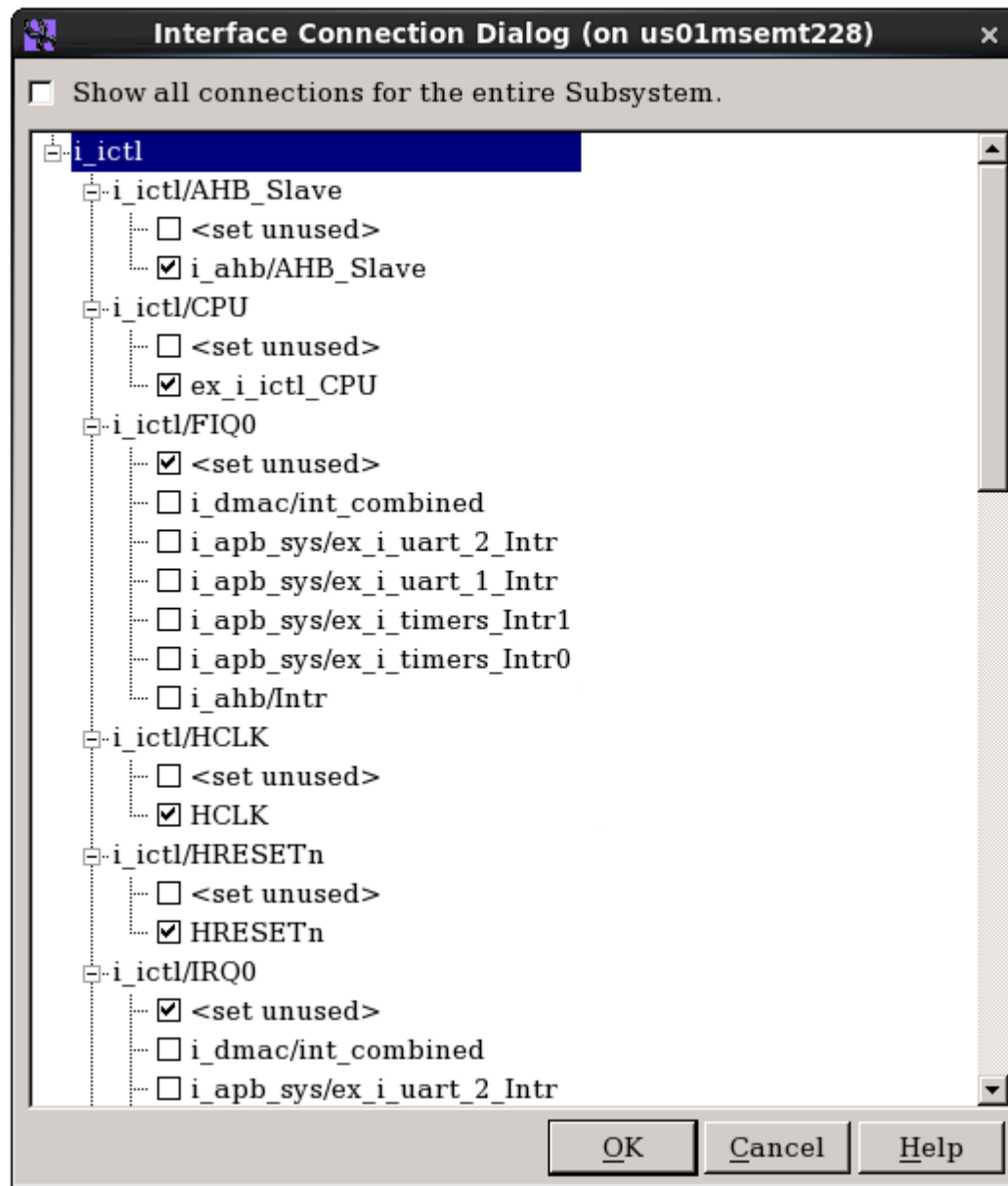
Notice that the new cell is outlined in red because some ports at this level need to be connected, which you will do next.

23. If you are not already in the schematic view, click the button before continuing as seen in [Figure 2-14](#).

Figure 2-14 Schematic Toggle Button



24. Select the i_ictl instance, right-click and choose **Change Connection**. The following dialog appears, where you can make these connections.

Figure 2-15 Interface Connection Dialog Window

Make the following connections for FIQ0 and IRQ0 through IRQ5 by checking the appropriate box for each of these interrupts:

- ☐ i_ictl/FIQ0 = i_dmac/int_combined
- ☐ i_ictl/IRQ0 = i_ahb/Intr
- ☐ i_ictl/IRQ1 = i_apb_sys/ex_i_uart_1_Intr
- ☐ i_ictl/IRQ2 = i_apb_sys/ex_i_uart_2_Intr
- ☐ i_ictl/IRQ3 = i_apb_sys/ex_i_timers_Intr0

- i_ictl/IRQ4 = i_apb_sys/ex_i_timers_Intr1
- i_ictl/IRQ5 (leave as <set unused> for now)

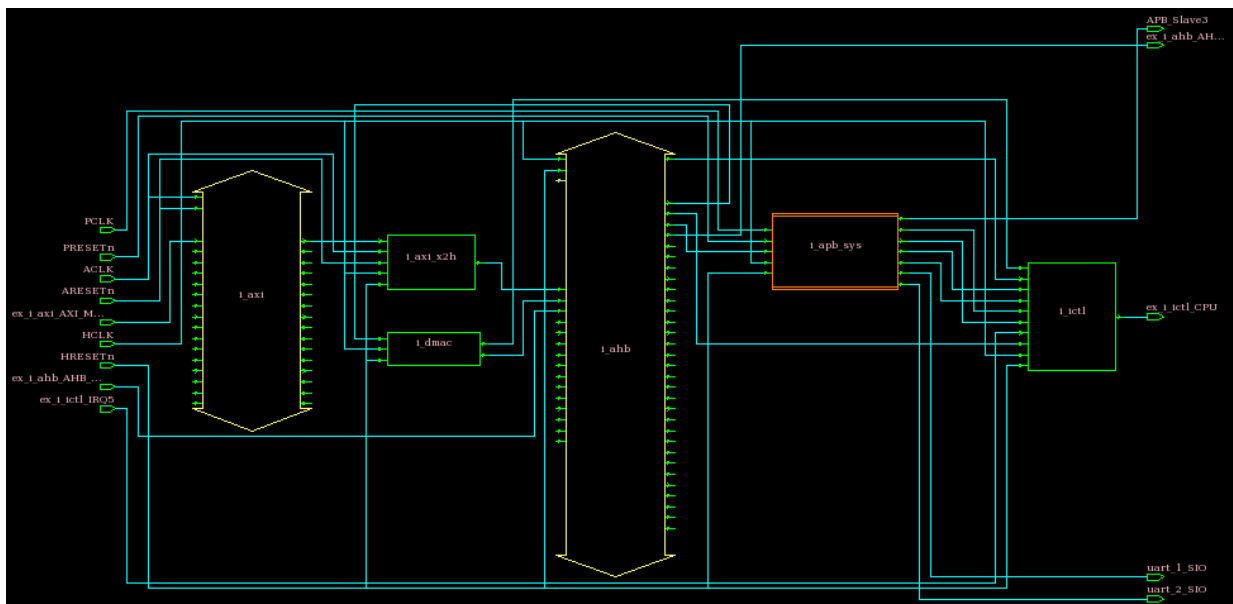
Click **OK** in the Interface Connections dialog to make these interrupt connections.

25. In the i_ictl instance, export the remaining unconnected IRQ5 off-chip (use default naming).
26. Finish the connections to the i_apb_sys block by exporting the remaining signals, naming them:
 - PCLK = PCLK
 - PRESETn = PRESETn
 - i_uart_1_SIO = uart_1_SIO
 - i_uart_2_SIO = uart_2_SIO
27. Finally, export these interface ports:
 - i_ahb/AHB_Master3 (default naming)
 - i_ahb/AHB_Slave4 (default naming)

28. On closer look at [Figure 2-1](#), we still need to export an APB Slave interface.
 - a. Double-click on the i_apb_sys hierarchical block to enter it again.
 - b. In the i_apb instance, select and export the APB_Slave3 interface port (using default naming).
 - c. Exit the cell and return to the top-level schematic. Note that the i_apb_sys instance now has a new port which needs to be connected.
 - d. Select and export this new port, naming it "APB_Slave3".

You have now added and configured interfaces for all the components in this subsystem. The top-level schematic looks similar to the schematic below.

Figure 2-16 Top Level Schematic Diagram



29. Although the coreAssembler tool performs automatic saves after each added component and when an activity successfully completes, explicitly save your workspace at this key point in the tutorial.

30. Choose the **Help > Check for IP Updates...** menu item to see if your IP is current. You will want to know this information and possibly make updates before you synthesize or simulate your design.

The IP Update Check report compares the components in your schematic to those in your DESIGNWARE_HOME tree, and also to the latest components that are available from Synopsys (via the Internet). Synopsys Technical Action Request (STAR) updates are also listed in this report to help you determine if you need to make an update. Viewing STARs and downloading components from Synopsys requires SolvNet authentication.

Figure 2-17 IP Update Check Report

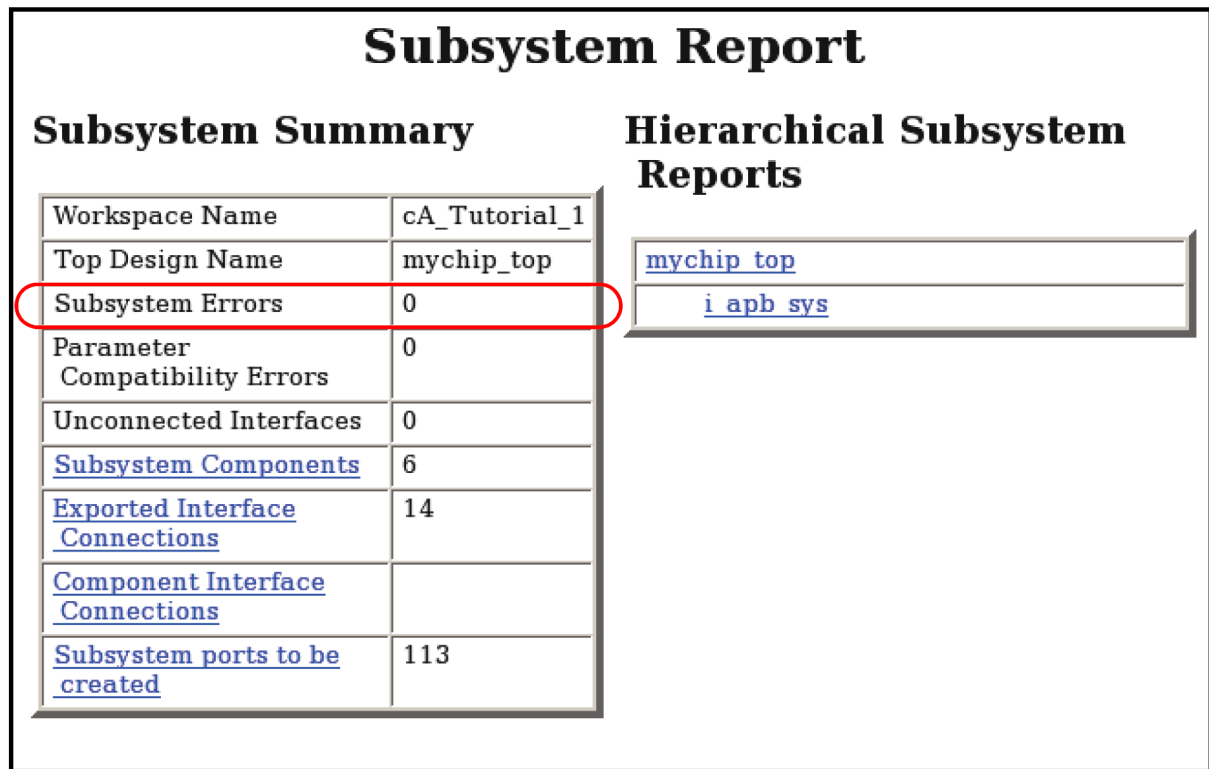
IP Update Check Report					
Report Date: 20/06/2019					
Component Name	Instance Name	Instantiated Version	Search Path Version	Latest Available Version	Reported Stars (New stars in bold)
DW_ahb_ictl	i_ictl	2.14a	2.14a	2.14a	No stars found Stars on the Web DW_ahb_ictl
DW_apb_uart	i_uart_1 i_uart_2	4.02a	4.02a	4.02a	No stars found Stars on the Web DW_apb_uart

The first time you use this feature, you might be prompted to enable automatic update checking, and to specify the interval between checks. You can change these preferences at any time using the **Edit > Preferences** menu item.

For more information on the IP Update Checking feature, refer to “Component Update Checking” in the *coreAssembler User Guide*.

31. Close the IP Update Check Report window.
32. Click on the **Report** tab (below the Activity View window) and, in the Subsystem Contents, click on **generate report** to create and view the Subsystem Report. Make sure that there are no subsystem errors.

Figure 2-18 Subsystem Report



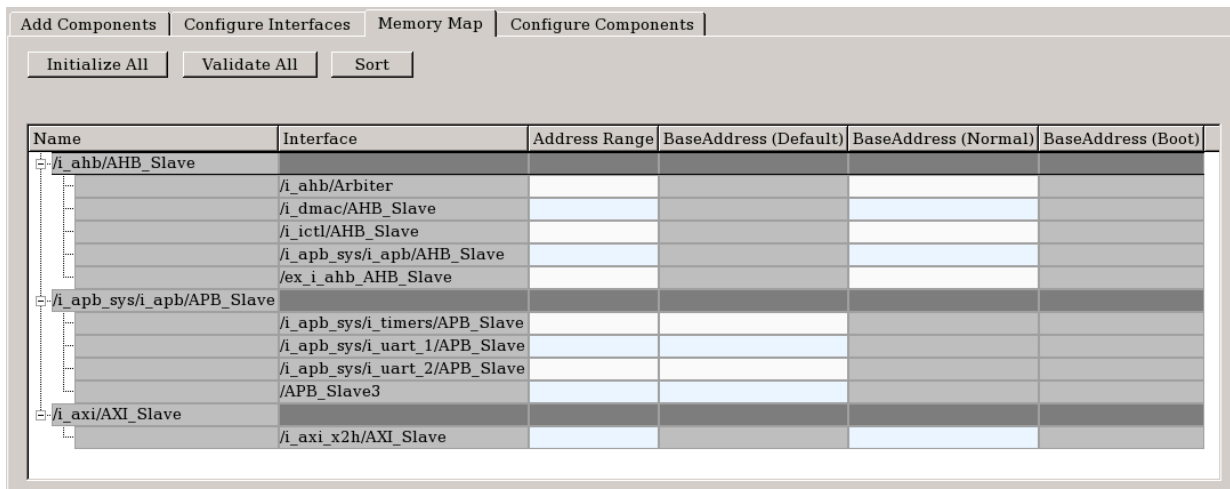
33. After viewing the report, click on the **Dialog** tab (next to the **Report** tab) to return to the editing environment.

2.7 Procedure 3: Specify Subsystem -- Memory Map Specification

1. Above the schematic view, click on the **Memory Map** activity tab. (If there is no Memory Map tab, use the **Edit > Workspace Options** menu and check the Show “Memory Map Specification” activity box.)

The Memory Map view appears, which allows you to initialize memory map values, customize those values, and validate the final memory map.

Figure 2-19 Specify Subsystem – Memory Map Specification

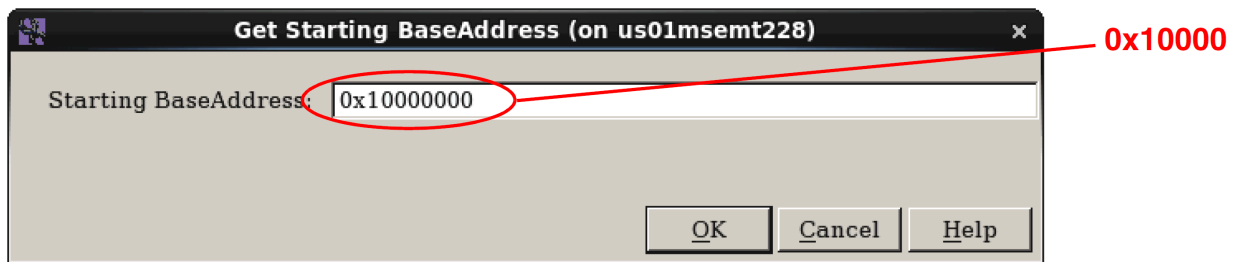


The screenshot shows the 'Memory Map' tab in a software interface. At the top, there are buttons for 'Add Components', 'Configure Interfaces', 'Memory Map' (selected), and 'Configure Components'. Below these are buttons for 'Initialize All', 'Validate All', and 'Sort'. The main area is a table with the following columns: Name, Interface, Address Range, BaseAddress (Default), BaseAddress (Normal), and BaseAddress (Boot). The table contains several rows of component names and their corresponding interfaces, with some cells highlighted in blue.

Name	Interface	Address Range	BaseAddress (Default)	BaseAddress (Normal)	BaseAddress (Boot)
/i_ahb/AHB_Slave					
	/i_ahb/Arbiter				
	/i_dmac/AHB_Slave				
	/i_ictl/AHB_Slave				
	/i_apb_sys/i_apb/AHB_Slave				
	/ex_i_ahb_AHB_Slave				
/i_apb_sys/i_apb/APB_Slave					
	/i_apb_sys/i_timers/APB_Slave				
	/i_apb_sys/i_uart_1/APB_Slave				
	/i_apb_sys/i_uart_2/APB_Slave				
	/APB_Slave3				
/i_axi/AXI_Slave					
	/i_axi_x2h/AXI_Slave				

2. Click the **Initialize All** button to auto-initialize the map.
You are prompted to provide a base address. Set it to **0x10000**.

Figure 2-20 BaseAddress Setup Window



The spreadsheet is filled with values based on the requirements for each of the components. You can manually change any of these values to fit your needs.

Figure 2-21 Component Spreadsheet Window

Name	Interface	Address Range	BaseAddress (Default)	BaseAddress (Normal)	BaseAddress (Boot)
-i_ahb/AHB_Slave					
	/i_ahb/Arbiter	0x1000		0x14000	
	/i_dmac/AHB_Slave	0x1000		0x15000	
	/i_ictl/AHB_Slave	0x1000		0x16000	
	/i_apb_sys/i_apb/AHB_Slave	0x4000		0x10000	
	/ex_i_ahb_AHB_Slave	0x1000		0x17000	
-i_apb_sys/i_apb/APB_Slave					
	/i_apb_sys/i_timers/APB_Slave	0x1000	0x10000		
	/i_apb_sys/i_uart_1/APB_Slave	0x1000	0x11000		
	/i_apb_sys/i_uart_2/APB_Slave	0x1000	0x12000		
	/APB_Slave3	0x1000	0x13000		
-i_axi/AXI_Slave					
	/i_axi_x2h/AXI_Slave	0x8000		0x10000	

**Note**

The i_apb_sys APB slaves are mapped into the AHB Slave address range (beginning at 10000, and each APB slave is given a range of 1000 in the AHB Slave space).

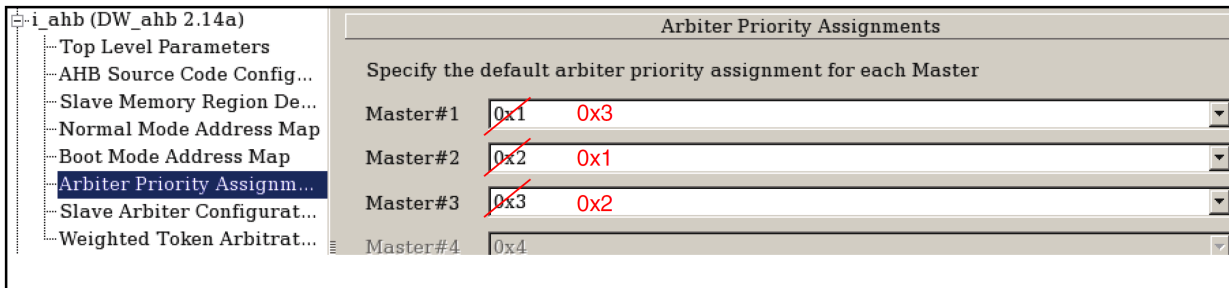
- The memory map in this tutorial procedure is valid, but to check if the values create a valid memory map, click on the **Validate All** button. Because no messages are reported, the memory map is valid.

In the next procedure, you will generate and view the Memory Map report.

2.8 Procedure 4: Specify Subsystem -- Configure Components

1. Click on the **Configure Components** activity tab. You will see the Apply Memory Map Changes dialog. Click OK to apply the changes that you made in the final step of the previous procedure.
2. The left pane shows a collapsible list of components that are configurable. Here, you will make several configuration changes.
3. Expand **i_ahb** (DW_ahb <version>) and click on **Arbiter Priority Assignments**. Change the assignments as follows:

Figure 2-22 Arbiter Priority Assignment Window



4. Again for the **i_ahb** instance in the left pane, click on the **Slave Arbiter Configuration** item and check the following boxes:
 - ☐ Use Hard-coded Arbiter Priorities?
 - ☐ Use Hard-coded Default Master?
5. In the left pane, expand the **i_apb_sys** subsystem and then expand **i_timers**. Click on **Timer 1 Configuration** and set the width of timer #1 to 16 bits.
6. Click on **Timer 2 Configuration** and set the width of timer #2 to 22 bits.
7. In the left pane, expand **i_axi** (DW_axi <vers>) and then expand **Arbitration Options**. Click on **Arbitration Type** then scroll down in the right pane and, for Slave Port 0 and Slave Port 1, set all **Priority** fields to "Fair Among Equals".
8. In the left pane, expand the **i_ictl** instance and click on **Top Level Parameters**. In the right pane, scroll down and in the **IRQ Configuration** section, check the box for **Install Priority Controller?**.
9. Again for the **i_ictl** instance in the left pane, click on **Priority Controller Configuration** and ensure the priority of the IRQs are set as follows:
 - ☐ Priority level of IRQ Source 0 = 0x0
 - ☐ Priority level of IRQ Source 1 = 0x3
 - ☐ Priority level of IRQ Source 2 = 0x2
 - ☐ Priority level of IRQ Source 3 = 0x1
 - ☐ Priority level of IRQ Source 4 = 0x4
 - ☐ Priority level of IRQ Source 5 = 0x5
10. Now, click **Apply** to complete the Specify Subsystem activity. If needed, you can return to the Specify Subsystem activity at any time to modify settings you made in the previous procedures.

The transcript shows the stages of execution for the coreAssembler tool as it elaborates the components and makes auto-connections. To this point, you have created a minimal design using the specified configuration and interface connections. When the coreAssembler tool completes this step, your view changes to the Report tab, where you have access to a number of reports about the design.

Also, if you look in the Activity List pane in the main window, you can see that the Specify Subsystem activity is now checked, showing it has been successfully completed.

11. From the Report tab, find the Memory Map entry in the Activity Specific Reports section and click **generate report**. This generates and displays the Memory Map Specification Report.

Figure 2-23 Memory Map Specification Report

Memory Map Specification Report			
Index By Master /i_dmac/AHB_Master1 /ex_i_ahb_AHB_Master			
Memory Map Diagrams /i_dmac/AHB_Master1 /ex_i_ahb_AHB_Master			
Index By Interface /i_axi/AXI_Slave /i_ahb/AHB_Slave /i_apb_sys/i_apb/APB_Slave			
Index By Mode Default Boot			
Memory map for /i_dmac/AHB_Master1			
Interface	Address Range	Base address (Normal)	Base address (Boot)
i_apb_sys/i_apb/AHB_Slave	0x4000	0x10000	
i_ahb/Arbiter	0x1000	0x14000	
i_dmac/AHB_Slave	0x1000	0x15000	

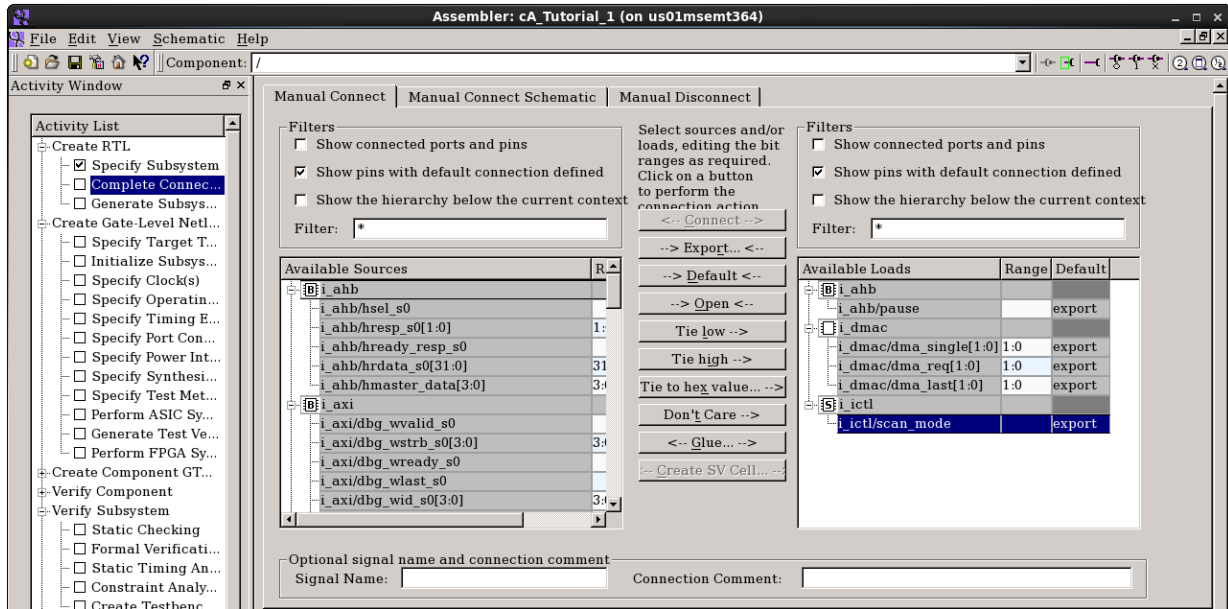
12. Above this report, click the back arrow to return to the report summary page, and then click on **generate report** for all of the available reports to generate and familiarize yourself with these reports. Navigate to lower report levels as necessary to get a good idea of the important information available.
13. Return to the Component Configuration report page and click on the **html** link for /i_ahb. As you scroll down in this report, note that entries shaded in gray are disabled in your current configuration.

If desired, you can use the **docbook** links to obtain DocBook formatted XML files of these reports for use in your component specifications and databooks.

2.9 Procedure 5: Complete Connections

1. In the Activity List pane on the left, click on the **Complete Connections** activity. The connections control panel appears in the view pane, as shown below:

Figure 2-24 Complete Connections Window



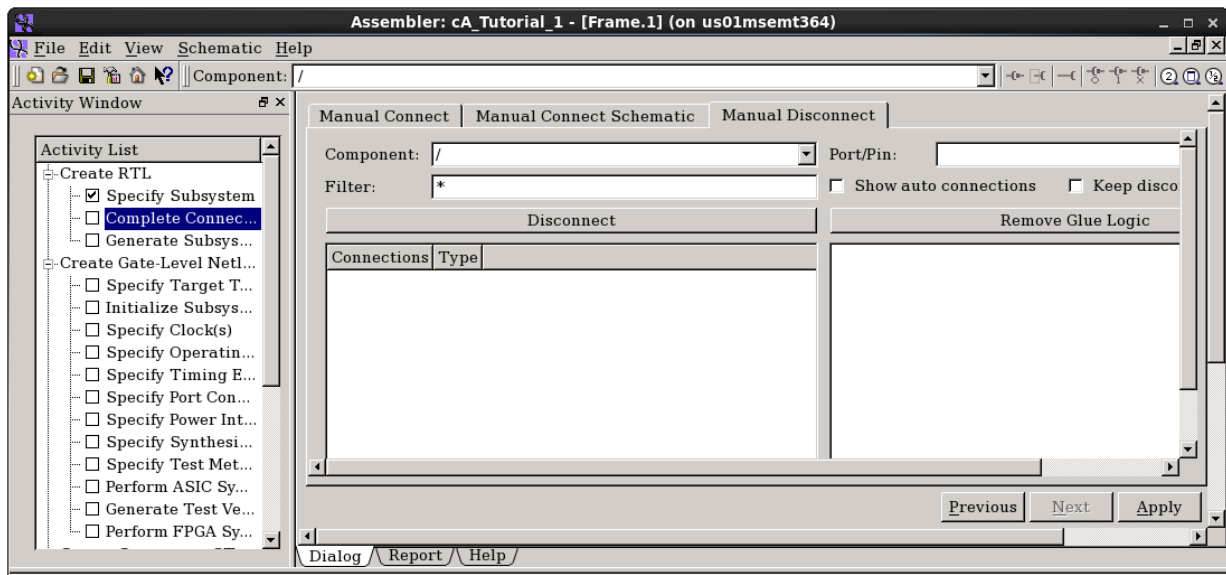
This dialog has separate tabs for “Manual Connect” and “Manual Disconnect” changes, and also a schematic view (“Manual Connect Schematic” tab) where you can make graphical connections.

Note that the Component drop-down in the Toolbar sets the hierarchical context at the top level.

2. In the right pane under Available Loads, select **i_ictl/scan_mode** and connect it to '0' by clicking the **Tie low** button (the Tie low button resides between the two panes).

The signal is removed from the window, indicating that it now has a valid connection.

3. Above the Available Loads pane, click the **Show the hierarchy below the current context** checkbox to view the /i_apb_sys hierarchy.
4. Set the following to '0' (Tie low). Use CTRL#click to select all these signals before clicking the Tie low button.
 - ☐ i_uart2/scan_mode
 - ☐ i_uart_1/scan_mode
 - ☐ i_timers/scan_mode
5. Click on the **Manual Disconnect** activity tab where you can navigate to and disconnect ports/pins that were manually connected.

Figure 2-25 Manual Disconnect Activity

The initial view begins at the top of the hierarchy, shown in the Component drop-down field as “/”. You use the Component drop-down need to traverse down the design hierarchy.

6. Use the Component drop#down to choose `i_ictl` and then ensure the Port/Pin drop#down shows `ictl/scan_mode`. In the left pane, click the Type field to select L (load). The load status appears in the right pane.
7. Click **Apply** to accept all of the connection changes you just made.

The view now shows the Report tab.

8. Click on the **generate report** link in the Connection reports area.
9. In the Report Link column, click on the **html** link for `mychip_top`.

This report shows you the Auto and Manual connections within your subsystem design, and the ports and signals that remain unconnected.

Figure 2-26 Connection Details Report for mychip_top

Connection Details Report for mychip_top

- [Automatic Connections](#)
- [All Connections](#)
- [Unconnected Ports](#)
- [Unconnected Pins](#)

Automatic Connections

Status: **Finished successfully.**

Auto-Created Subsystem Ports

Direction	Name	Range	Interface Port
input	ACLK_ac1k	<i>single bit</i>	ACLK/ac1k
output	APB_Slave3_paddr	31 : 0	APB_Slave3/paddr
output	APB_Slave3_penable	<i>single bit</i>	APB_Slave3/penable
input	APB_Slave3_prdata	31 : 0	APB_Slave3/prdata
output	APB_Slave3_psel	<i>single bit</i>	APB_Slave3/psel
output	APB_Slave3_pwdata	31 : 0	APB_Slave3/pwdata
output	APB_Slave3_pwrite	<i>single bit</i>	APB_Slave3/pwrite
input	ARESETn_aresetn	<i>single bit</i>	ARESETn/aresetn
input	ex_i_ahb_AHB_Master_haddr	31 : 0	ex_i_ahb_AHB_Master/haddr
input	ex_i_ahb_AHB_Master_hburst	2 : 0	ex_i_ahb_AHB_Master/hburst
input	ex_i_ahb_AHB_Master_hbusreq	<i>single bit</i>	ex_i_ahb_AHB_Master/hbusreq
output	ex_i_ahb_AHB_Master_hgrant	<i>single bit</i>	ex_i_ahb_AHB_Master/hgrant
input	ex_i_ahb_AHB_Master_hlock	<i>single bit</i>	ex_i_ahb_AHB_Master/hlock
input	ex_i_ahb_AHB_Master_hprot	3 : 0	ex_i_ahb_AHB_Master/hprot

10. Click on the links at the top to familiarize yourself with the various connection reports.

2.10 Procedure 6: Generate Subsystem RTL

1. In the Activity List, click on the **Generate Subsystem RTL** activity. In the Activity window, the following choices are presented:

Figure 2-27 Activity List Window

Output language: Verilog

Preferred VHDL logic type: ☒ std_logic ☐ std_ulogic

Include testbench probe signals: ☐

Write System Verilog `include files: ☐

Use compact instance syntax: ☒

Include modport in interface connections: ☐

Include port descriptions in RTL: ☐

Signal name format: %c_%p

Use unique logic nets: ☐

Use nets for open output pins: ☐

Use assign for initialization: ☐

Allow reading of output ports: ☐

Don't Write Synopsys Header: ☐

Allow Aggregate Assignments: ☒

Don't tie open inputs to logic X: ☐

Define Wires for all Ports: ☐

Write bit vector literals in binary format: ☐

User comment inserted into each generated file:

Component	VHDL Library	Vendor
/	work	ThirdParty
/i_apb_sys	work	ThirdParty

- ☐ Create the top-level design in Verilog, VHDL, System Verilog, or Verilog 1995 (the default here is determined by the “Output language” setting in the Create Workspace dialog box, which you set in [“Procedure 1: Create a New Workspace”](#) on page 24).
 - ☐ If you choose VHDL, you can assign a VHDL Library for each level, preferred logic type, and probes.
 - ☐ You can also add a comment that will be inserted into the generated files.
2. Use all defaults and add the comment: “Synopsys coreAssembler Tutorial 1: <date>”
 3. Click **Apply** to create the design RTL.

When the processing is complete, the Activity view changes to the Report tab.

4. Click on the **Subsystem Source Code** link to view the generated files. The view for Verilog is:

Figure 2-28 Report and Generated Files**Generated Subsystem**

Workspace Name	cA_Tutorial_1
Top Design Name	mychip_top
Language	verilog
Date	Fri Jun 21 12:20:41 PDT 2019

Hierarchical Subsystem Reports

mychip_top
i_apb_sys

Generated Files

Name	Description	Generated
src/mychip_top.v	Verilog top-level subsystem RTL.	06/21/2019 12:20:29
export/mychip_top.lst	Simulation file list	06/21/2019 12:20:40
export/mychip_top_sim.lst	Simulation file list	06/21/2019 12:20:40

5. Under Generated Files, click on the link to view the top-level source file ([src/mychip_top.v](#)).
6. Choose **File > Save Workspace** to save your completed RTL design.
7. Choose **File > Write Batch Script** to create a batch script that can be used to recreate the RTL design.
 - a. Enter the Output file name: "cA_Tutorial_1_batch.tcl"
 - b. Check the box to Write default parameter values. OK the dialog box.

The batch script is created in the current working directory, or at the location you specify. This step can take some time.

8. Open the batch script file that you just created and view the file structure and contents.

In the header, notice the command that was used to create this batch file is listed. Also note the "create_workspace" command (first command below the header) that is used to create the new workspace when this script is run.

9. Close this batch script file.
10. Choose **File > Exit** to close coreAssembler, and save the workspace if needed.

Congratulations! You have just completed Tutorial 1. In this tutorial, you:

- Invoked coreAssembler and created a new workspace
- Created a hierarchical level containing the APB subsystem
- Added components which used automatic bus connections to create your subsystem
- Exported AHB/AXI master and AHB/APB slave interfaces
- Created a memory map
- Configured instances of components within your subsystem design
- Made manual connections within your subsystem design
- Created a batch script for recreating your subsystem

3

Tutorial 2: Creating a Testbench

This chapter contains the following sections:

- [“Description”](#) on page 48
- [“Objective”](#) on page 49
- [“Before You Begin”](#) on page 50
- [“Procedure 1: Generate a Workspace from a Batch Script”](#) on page 51
- [“Procedure 2: Creating a UVM \(RAL Subsystem\) Testbench Workspace”](#) on page 52

3.1 Description

In this tutorial, you will create a UVM testbench. The UVM testbench consists of the following Synopsys Verification IP (VIP) models:

- TB_clkgen
- SvtAhbMasterVip
- SvtAxiMasterVip

The testbench performs the following tests for each slave component that has a memory map defined:

- Hardware reset
- Bit-bash
- Register access tests

3.2 Objective

The purpose of this tutorial is to show the creation of a testbench in coreAssembler. For this tutorial, you will go straight to the Create Testbench Workspace and Simulate Subsystem Testbench activity. The following procedures are explained:

- [“Procedure 1: Generate a Workspace from a Batch Script”](#) on page 51
- [“Procedure 2: Creating a UVM \(RAL Subsystem\) Testbench Workspace”](#) on page 52

3.3 Before You Begin

Because the tutorials are streamlined to help you complete them in minimum time, some of the general information about the coreAssembler tool (or other tools) is not provided in the tutorial flow. Instead, links are provided to these documents:

- *coreAssembler User Guide*
- *coreConsultant User Guide*

3.4 Procedure 1: Generate a Workspace from a Batch Script

A batch script is an easy way to save and recreate your design workspace. When you completed Tutorial 1, you saved a batch script for your subsystem. In this procedure, you will create a new workspace using this saved batch file.

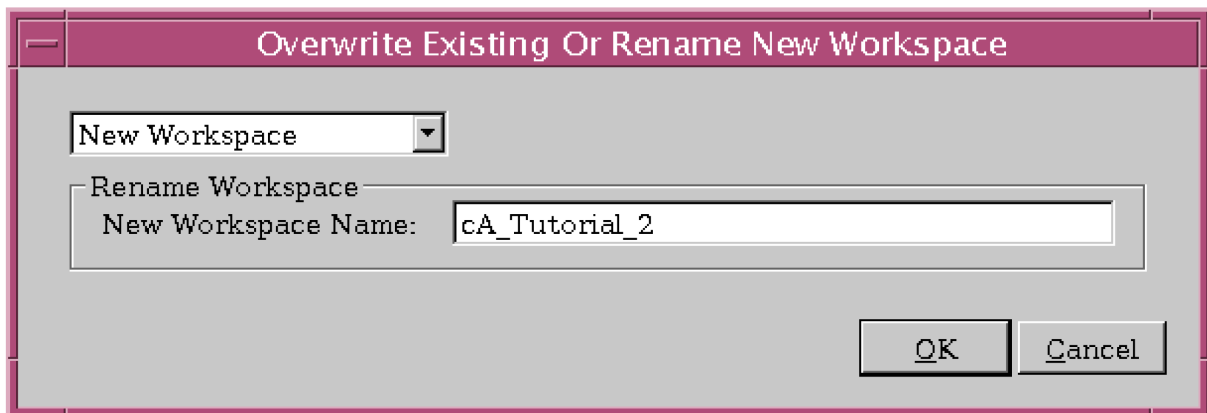
1. Invoke the coreAssembler tool if it is not already open. Or if you have just finished Tutorial 1, you can click on the home toolbar icon to return to the coreAssembler home page.
2. If not already closed, be sure to save and close the cA_Tutorial_1 workspace.
3. In the coreAssembler command line, issue the following command to run the batch script:

```
coreAssembler> source cA_Tutorial_1_batch.tcl
```

Because the “cA_Tutorial_1 workspace already exists, you are asked if you want to “Overwrite Existing Or Rename New Workspace”.

4. Use the drop-down box to change Overwrite to **New Workspace** and enter **cA_Tutorial_2** as the new workspace name, and then OK the dialog box.

Figure 3-1 New Workspace Window



5. The script runs, and creates the new cA_Tutorial_2 workspace, leaving you at the last activity that you completed in Tutorial 1.
6. Save the cA_Tutorial_2 workspace.



You can also run the batch script in non-GUI mode using the command:

```
% coreAssembler -shell -f cA_Tutorial_1_batch.tcl
```

But first you must open the file and change the target name (cA_Tutorial_1) for the create_workspace command to “cA_Tutorial_2” and then save the file.

Next, you will use the subsystem RTL in the cA_Tutorial_2 workspace to create a testbench to simulate your design.

3.5 Procedure 2: Creating a UVM (RAL Subsystem) Testbench Workspace

In this procedure, you will simulate the entire subsystem using the UVM environment.

1. If the `cA_Tutorial_2` workspace is not open in the coreAssembler tool, open it by using the command,

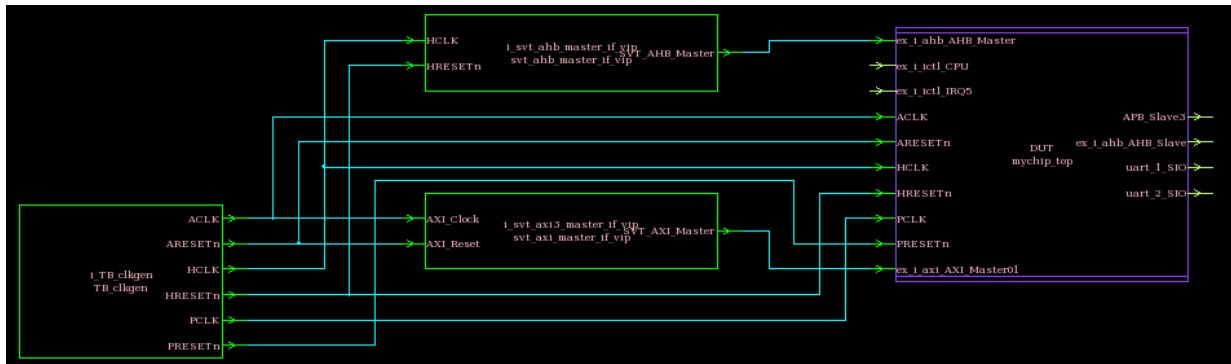
```
% coreAssembler cA_Tutorial_2.
```

2. In the Verify Subsystem section of the Activity List, click on **Create Testbench Workspace**.
3. In the Activity window, set the following values and click **Apply**.

- Testbench Workspace Name = **tb_cA_Tutorial_2b**
- Testbench Strategy and Language = **UVM RAL Subsystem (default)/ UVM Subsystem**

The coreAssembler tool creates a new testbench with UVM models and a clock generator, as shown below. Notice that a Clock Gen block (`i_TB_clkgen`) is automatically added. Also notice that the UART outputs are not connected to the SIO VIP because the SIO verification models are not available for the UVM environment.

Figure 3-2 TestBench Diagram



4. Again, if the DUT is outlined in red, resolve all port connections (export, set unused, or tie low/high). When all port connections are made, the DUT is outlined in purple.
5. In the Activity List, click on the **Simulate Subsystem Testbench** activity. Click **Yes** when you are asked if you want to auto-complete all of the testbench creation activities. Also, check if VCS has been installed.

The coreAssembler tool then automatically completes all the activities for Create Testbench, as shown by the check#marked items in the Activity List. Note that you can optionally create reports for each completed activity using the File > Generate Reports menu item.

When Simulate Subsystem Testbench is the current activity, notice that VCS is the only simulator choice for a UVM simulation.

6. Click the **Execution Options** tab, and check the **Generate scripts only?** option. Click **Apply** to generate the scripts.
7. When the activity completes, open the **Simulation Build and Run Summary** report. At the top of the report, notice that the Job Status says "Script(s) generated, but NOT run."
8. Save your testbench workspace, and exit coreAssembler.

9. In your UNIX environment open a new term window and navigate to the `tb_cA_Tutorial_2b/sim` directory and view the contents of the `run.scr` file. This file launches the batch simulation run.
10. Before you run the simulation script, ensure you have set the tool root for VCS and Design Compiler. To set the tool root paths, use the `setenv` command as follows:

```
% setenv SYNOPSYS <path_to_DC>  
% setenv VCS_HOME <path_to_VCS>
```
11. Run the simulation by executing this file: `run.scr`

3.5.1 Tutorial 2 Summary

Congratulations! You have just completed Tutorial 2. In this tutorial, you:

- Created a UVM Testbench
- Generated simulation scripts
- Performed the UVM simulation using the simulation scripts

4

Tutorial 3: Synthesizing Your Design

This chapter contains the following sections:

- [“Description”](#) on page 56
- [“Objective”](#) on page 57
- [“Procedure 1: Recreating a Workspace from Batch”](#) on page 58
- [“Tutorial 3 Summary”](#) on page 65

4.1 Description

This tutorial familiarizes you with the Synthesis activities within coreAssembler. It shows the activities you use to set up for synthesis, and presents the pre-defined strategies and strategy options you use to perform the synthesis run.

4.2 Objective

For this tutorial, you will perform the following tasks:

- Recreate the subsystem you created in Tutorial 1 using a batch script
- View synthesis setup activities for the subsystem using the default settings
- Auto-complete some synthesis activities using defaults
- Synthesize the subsystem using a pre-defined strategy for Design Compiler

4.3 Procedure 1: Recreating a Workspace from Batch

A batch script is an easy way to save and recreate your design workspace. When you completed Tutorial 1, you saved a batch script for your subsystem. In this procedure, you will create a new workspace using this saved batch file.

1. Set your working directory to the directory containing the `cA_Tutorial_1_batch.tcl`.
2. Invoke coreAssembler if it's not already open.
3. In the command line field (the **coreAssembler** > prompt) at the bottom of the window, issue the following commands:

```
> source cA_Tutorial_1_batch.tcl
```

4. When prompted, change the workspace name as "cA_Tutorial_3" and **OK** the dialog.

This step takes several minutes, and refreshes your view as the process continues.

You are now going to perform synthesis on the subsystem. But first you can streamline coreAssembler processing by removing unneeded processing steps.

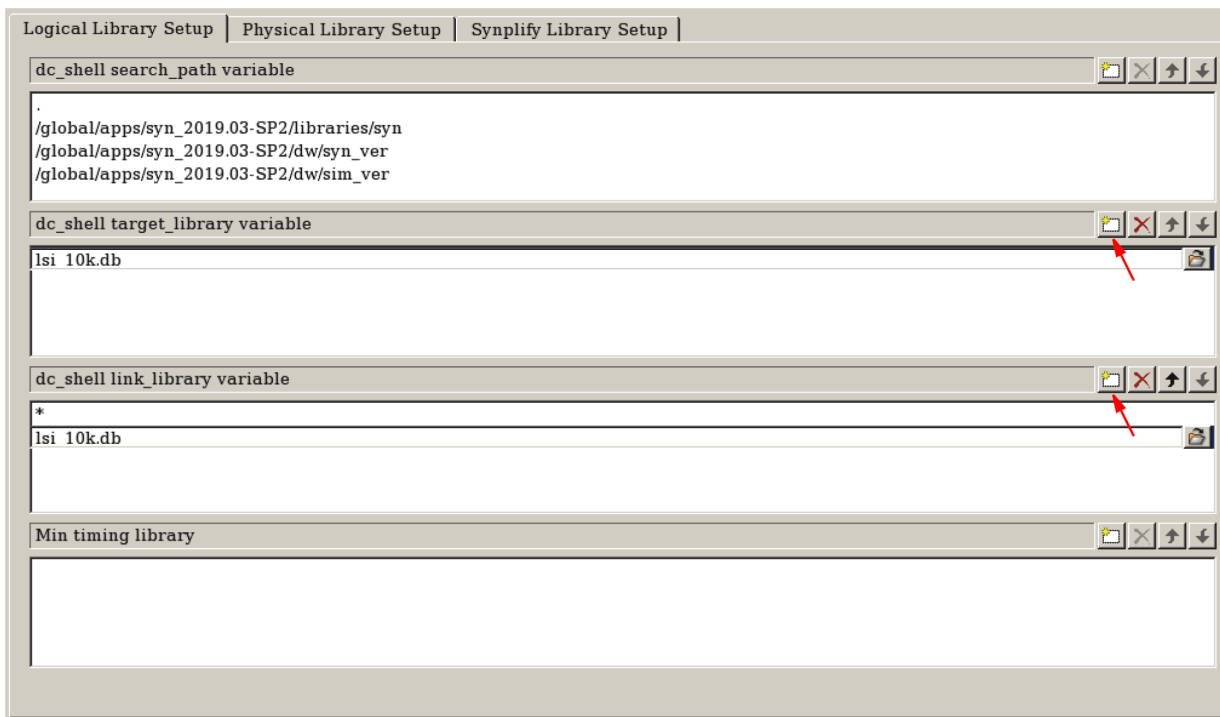
5. In the **Create Gate-Level Netlist** activity group, click on the **Specify Target Technology** activity.

The Specify Target Technology activity specifies the technology libraries for ASIC and/or FPGA synthesis. When you select this activity, the Library Setup tabs are displayed. Prior to showing this pane for the first time, your setup file (`.synopsys_dc.setup`) is read to determine initial values for the fields.

The following figure shows library setup information for an environment that has the current DesignWare Building Blocks (DWBB) Library installed. The red arrows point out how to add libraries for each category. Click the button as seen in [Figure 4-1](#) to select the "lsi_10k.db" library, as shown below. You do not need to specify a "Min timing library" for this tutorial.

Figure 4-1 Add New Button



Figure 4-2 Library Setup Information

You will use the default synthesis settings for this tutorial, so the coreAssembler tool will auto-complete most of the activities in the Create Gate-Level Netlist activity group.

6. Click **Apply** for this activity.
7. When the Specify Target Technology summary page appears, click the Implementation Technology report link; the Technology Report appears.

Figure 4-3 Technology Report

Technology Report

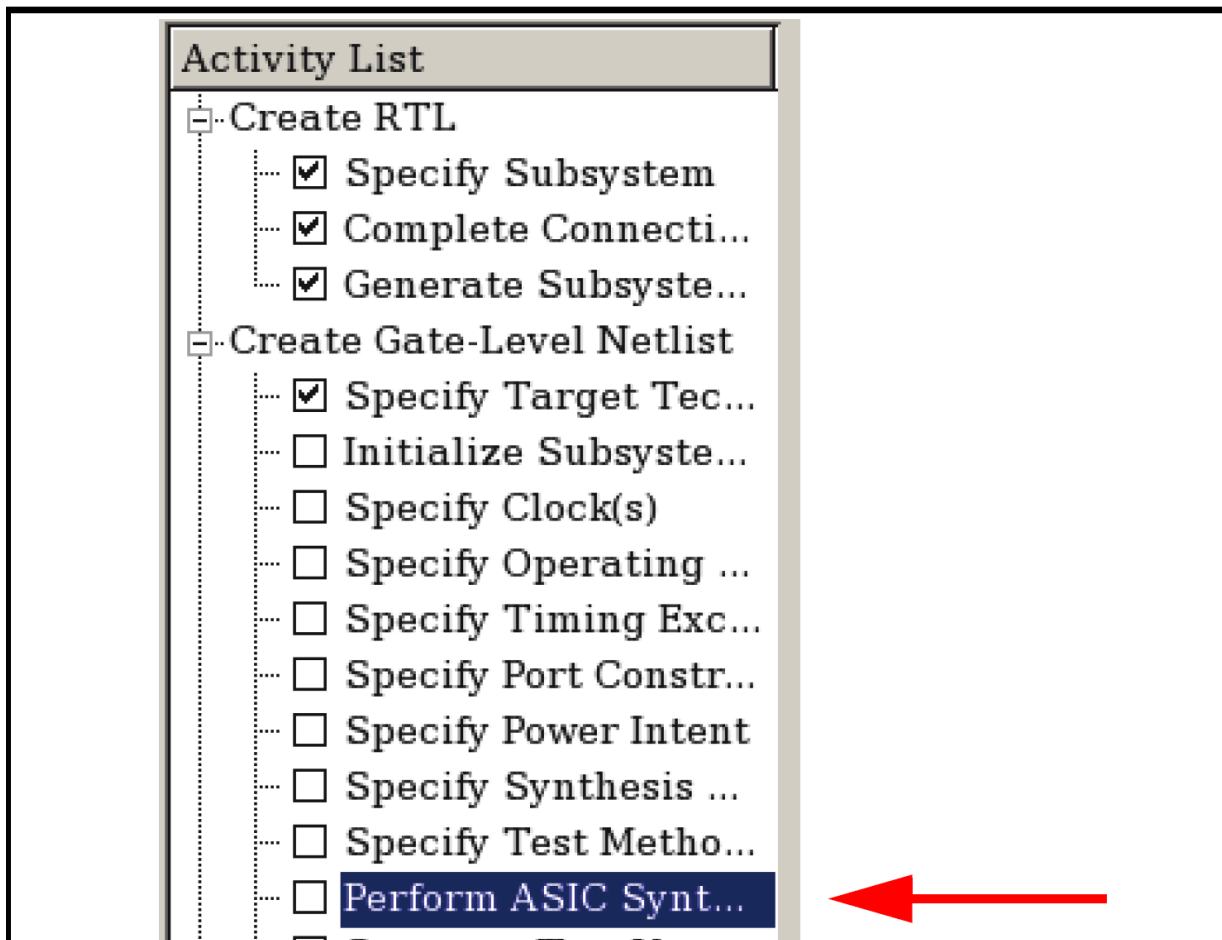
DC Technology Setup

Parameter	Value
search_path	. /global/apps/syn_2019.03-SP2/libraries/syn /global/apps/syn_2019.03-SP2/dw/syn_ver /global/apps/syn_2019.03-SP2/dw/sim_ver
target_library	lsi_10k.db
link_library	* lsi_10k.db
Min timing library	
Milkyway Design Library	
TechFileFormat	tf
TechFile	
CreateMwDesignOptions	
MaxTluPlusFile	
MinTluPlusFile	
Tech2ItfMap	
MaxEmulationTluPlus	
MinEmulationTluPlus	
Milkyway Reference Library	
Milkyway Logic 0 Net	VSS

For more information about the synthesis activities in coreAssembler, refer to the *coreAssembler User Guide* (“Creating the Gate-Level Netlist for a Subsystem” chapter). This manual can also be accessed from **Help > coreAssembler Tool Help > User’s Guide**. To obtain basic definitions for any of the activity parameters in the Create Gate#Level Netlist activity group, click on the Help tab for the specific activity window.

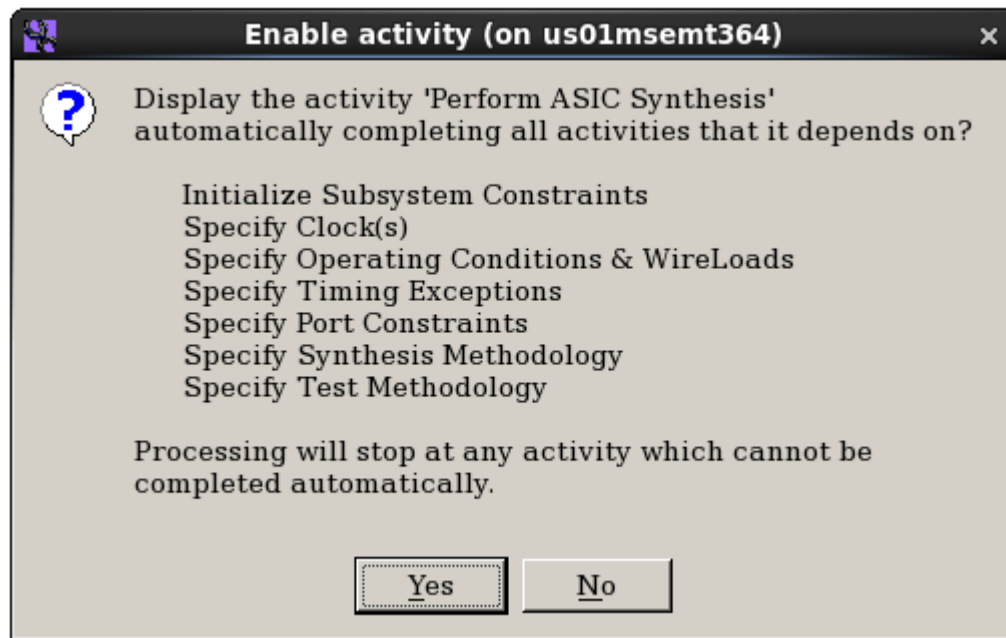
Also, the synthesis flow is documented in each Synopsys IP databook. You can access the databook of any DW component that you include in your subsystem from the coreAssembler **Help** menu.

- Click on the **Perform ASIC Synthesis** activity, shown below.

Figure 4-4 Activity List – Perform ASIC Synthesis

9. You are prompted to complete the remaining activities in the **Create Gate#Level Netlist** activity group. Click **Yes**.

Figure 4-5 Enable Activity Window

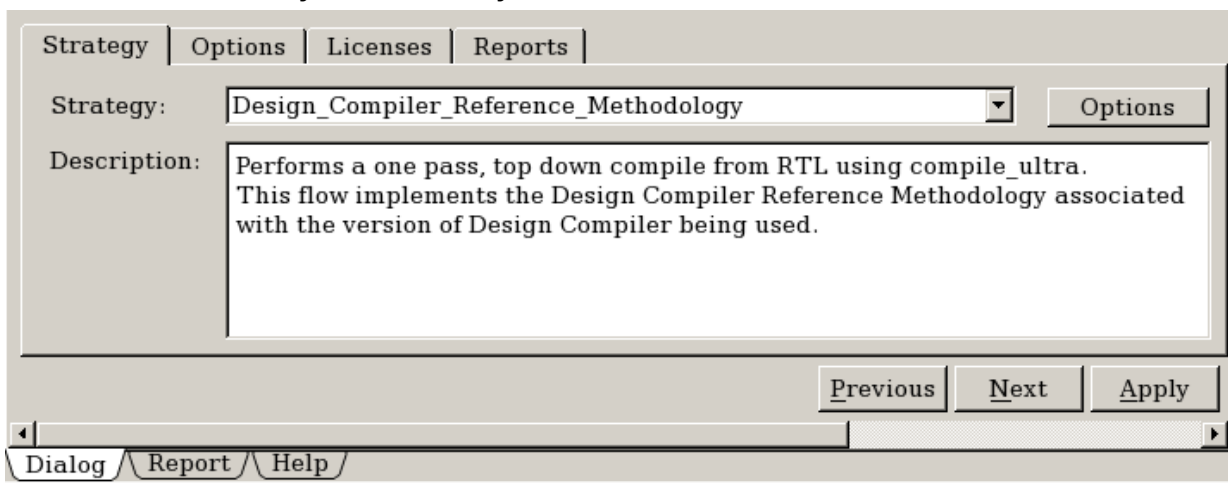


Depending on your Technology Library, you may be prompted to specify Operating Conditions. For example, you may have to specify a value for the OperatingConditionsWorst such as **WCCOM**.

You may also pause with a Wireloads Report, in which case, click the “Perform ASIC Synthesis” activity again and Yes to finish automatic completion.

The **Perform ASIC Synthesis** activity window is displayed and becomes active when the auto#completion process has finished. Here you choose the Synthesis strategy.

Figure 4-6 Perform ASIC Synthesis Activity Window



- Click the pull-down arrow on the Strategy field and examine the choices; each is described in the *coreAssembler User Guide*.

11. Examine the controls available to you on each of the tabs:
 - ❑ **Options** (generate scripts only, run style, notification)
 - ❑ **Licenses** (indicate which licenses can be used during the Synthesis run)
 - ❑ **Reports** (indicate which reports you will gather data for)
12. To the right of the Strategy field, click on the **Options** button (not the Options tab). This button displays several sets of Strategy Parameters that allow for detailed control of the synthesis run. We will use the default parameter values, so click Cancel or OK.

Before launching a synthesis run, you may want to only “generate scripts,” so that you can control the run yourself using the <workspace>/syn/run.scr/Makefile. If so, first choose this option from the **Options** tab.
13. In the **Perform ASIC Synthesis** activity window pane, click **Apply** to accept the default (Design_Compiler_Reference_Methodology) settings and begin the synthesis run.

After several minutes, the Synthesize activity is marked completed. This *does not* mean that the synthesis run has completed, just that the steps to launch the run have completed. Synthesis runs in the background and the window automatically refreshes its “Job Status” until synthesis has finished.
14. Click on the link to the **Synthesis Results** report.

Figure 4-7 coreAssembler Synthesis Results

Job Status

Run Style	Execution Host	Job Id	Job Status	Results File
local	us01msemt364.internal.synopsys.com	58347	running (kill)	report/synthesis.html

Last Update To Page: Fri Jun 21 17:46:04 2019



coreAssembler Synthesis Results

Compile Summary

Workspace: /slowfs/us01dwt2p272/gansberg/coreTools_work/P-2019.06/cA_Tutorial_3

Run Status: Started at Fri Jun 21 17:36:49 2019

[Log File Summary](#)
[Completed Synthesis Stages](#)
[Legend](#)

Log File Summary

Log file	Error Count	Description
initial.log	0	dc_shell log for stage 'initial'

After you have synthesized the entire subsystem the synthesis results can be found beneath the workspace directory in the “syn” folder. You can find the gate-level netlist (db or Verilog) in the *workspace/syn/final/db* directory. For a description of the files that are created in the “syn” folder, see the *coreAssembler User Guide*.

15. This marks the end of the tutorial, so exit the coreAssembler tool and save your workspace as you exit.

4.4 Tutorial 3 Summary

Congratulations! You have just completed Tutorial 3. In this tutorial, you:

- Created a workspace from a batch file
- Examined the libraries specified in the **Specify Target Technology** activity
- Specified synthesis constraints, clocks, and timing, using defaults
- Examined the synthesis options and strategies for the **Synthesize** activity
- Synthesized the subsystem
- Saved your results, and examined the data created

5

Menus and Toolbars

Table 5-1 lists toolbar icons and the corresponding menu items and commands to assemble a subsystem.

Table 5-1 coreAssembler Menu and Toolbar Options

Toolbar Icon	Menu Item	Description	Command Line Mode
File Menu			
None	Install coreKit	Install a coreKit Note: Synopsys synthesizable IP for AMBA 2/AMBA 3 AXI does not require installation.	batch_install
	New Workspace	Create a new coreAssembler generic workspace (not generally used in coreAssembler) To create an AMBA-specific coreAssembler workspace, use the create a new AMBA subsystem now link on the home page.	create_workspace
	Open Workspace	Open an existing workspace	open_workspace
	Save Workspace	Save the currently loaded workspace	save_workspace
None	Save Workspace As	Save the currently loaded workspace into a new directory	duplicate_workspace
None	Close Workspace	Close the currently loaded workspace	close_workspace
None	Generate Reports	Select and generate activity reports (reports are not automatically generated)	generate_reports

Table 5-1 coreAssembler Menu and Toolbar Options (continued)

Toolbar Icon	Menu Item	Description	Command Line Mode
None	Generate Optional Views	Select and generate alternate design views (alternate design views are not automatically generated)	generate_views
None	Write Batch Script	Create a TCL batch script to re-create the current state of your workspace	write_batch_script
None	Build Debug Tar-file	Generate debug info for Synopsys support Creates file named: <design>_debug_tar.gz	build_debug_tarfile
None	Write IP-XACT File	Export the current subsystem using IP-XACT XML file format	write_ipxact_component write_ipxact_design
None	Read IP-XACT File	Import an IP-XACT formatted XML file for use in coreAssembler	read_ipxact_file
None	Remove IP-XACT File	Remove an IP-XACT generators and other elements from coreAssembler	remove_ipxact_file
None	Tables... > Read Attribute Table > Write Attribute Table > Read Pin Connection Table > Write Pin Connection Table > Read Subsystem Table > Write Subsystem Table	Submenu of read/write table commands allowing table-driven data entry	read_attribute_table write_attribute_table read_pin_connection_table write_pin_connection_table read_subsystem_table write_subsystem_table
None	Exit	Exit coreAssembler	exit or quit
Edit Menu			
	Tool Installation Roots	Specify tool installation directories	set_tool_root
None	Preferences	Specify GUI preferences and some verification, execution, synthesis, and IP-XACT options	N/A
None	Workspace Options	Customize which activities and functionality are available in the current workspace	set_workspace_options
View Menu			
	Home Page	Show the coreAssembler Home Page in the Activity View pane	N/A

Table 5-1 coreAssembler Menu and Toolbar Options (continued)

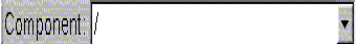
Toolbar Icon	Menu Item	Description	Command Line Mode
None	Reports	View a list of the available reports for the current subsystem	N/A
None	Dialog Window Console Window Activity Window Application Toolbar Schematic Toolbar	Show or hide the toolbars and windows Check marks indicate items that are visible.	N/A
Schematic Menu (visible when a workspace is open)			
	Add New Component	Add component instance to subsystem; also, insert glue logic	instantiate_component
None	Add Hierarchical Level	Add hierarchical cell to subsystem	create_hierarchical_component
None	Group components	Move selected components into a hierarchical cell, preserving connections	group_components
None	Ungroup component	Add hierarchical cell to subsystem	ungroup_component
None	Change Views	Changes view in multi-view IP-XACT XML component	set_component_view get_component_view
None	Import Component	Import an unpackaged component	import_component
	Duplicate Component	Duplicate selected component	duplicate_component
	Remove Item	Remove selected component or interface from subsystem	remove_component remove_exported_interface
	Replace Component	Replace selected component	replace_component
None	Attach Interfaces	Attach interfaces to the selected component	attach_interface
None	Rename	Rename selected component or exported interface	rename_component rename_exported_interface
None	Enter Cell Exit Cell	Navigate into and out of hierarchical components; also, you can also use the pull-down field in the toolbar: 	N/A

Table 5-1 coreAssembler Menu and Toolbar Options (continued)

Toolbar Icon	Menu Item	Description	Command Line Mode
		For more information about adding hierarchical components to your design, see the “New Hierarchical Component” section in the <i>coreAssembler User Guide</i> .	
None	Edit Search Path	Edit component search path	N/A
None	Load Interface Definitions	Loads an interface definition file	load_interface_definitions
None	Select Items	Select cells and interfaces using a convenient interface	N/A
	Change Connection	Change selected component/interface connection.	N/A
	Export Interface	Export selected interface	export_interface
None	Export all Unconnected Interfaces	Export any unconnected interface that requires a connection	N/A
	Set Unused	Set selected interface to unused	set_unused_interface
None	Mark All Unconnected Interfaces as Unused	Globally “set unused” for each unconnected interface	N/A
None	Detach Interface	Remove an attached interface from the given component	detach_interface
None	Port Associations	Associate and setup interface ports This is enabled only when you have selected an exported interface or an interface attached to an imported component.	N/A
	Filter	Filter displayed interfaces (Schematic View)	N/A
None	Propagate Memory Map	Propagate the memory maps visible from an exported slave interface to that slave	propagate_memory_map
None	Remove Memory Map	Remove a memory map	remove_memory_map
	Inspect > Toggle View	Toggle Activity View pane between Schematic and Tree views	N/A
	Inspect > Inspect Component	Open Component Inspector	N/A

Table 5-1 coreAssembler Menu and Toolbar Options (continued)

Toolbar Icon	Menu Item	Description	Command Line Mode
	Inspect > Component List	Open Component List Inspector	N/A
	Inspect > Exported Interface List	Open Exported Interface List Inspector	N/A
None	Open All	Expand all list items (Tree View)	N/A
None	Close All	Close all list items (Tree View)	N/A
None	Close All Components	Close all component list items (Tree View)	N/A
None	Move > Move Item Up Move > Move Item Down	Move list item up list (Tree View) Move list item down list (Tree View)	N/A
	Zoom > Zoom In 2X Zoom > Zoom Full Zoom > Zoom Out 2X	Zoom view in 2x (Schematic View) Zoom view to fit all (Schematic View) Zoom view out 2x (Schematic View)	N/A
Help Menu			
None	About	Displays the coreAssembler initial window	N/A
None	Check for IP Updates	Performs an IP check against the DesignWare Home Library, and against the latest IP on the Synopsys website	N/A
None	Show IP Update Report	Displays the most recent IP check report	N/A
None	coreAssembler Tool Help > Error Messages > User's Guide > Command Reference > Tutorial > Release Notes > Application Notes	Provides links to the various online manuals related to coreAssembler	N/A
None	Help for component / <instance_name> Support Info for <cmp> Data Book Release Notes Check Environment	Provides access to documentation for each of the DESIGNWARE_HOME components used in your subsystem Check Environment performs a tool version check (should be run before using simulation or synthesis)	N/A

Table 5-1 coreAssembler Menu and Toolbar Options (continued)

Toolbar Icon	Menu Item	Description	Command Line Mode
Icon Only Reference			
	N/A	Turns the cursor into an arrow with a question mark that allows you to obtain “What’s This?” help when you right-click on an item	N/A